
0.5um Digital CMOS 2P3M 5V Design Basic Tutorial

Version 1.0

National Semiconductor Public Laboratory (NSPL)

- Electronics and Telecommunications Research Institute (ETRI)
- Seoul National University (SNU)
- Daegu Gyeongbuk Institute of Science & Technology (DGIST)



서울대학교
SEOUL NATIONAL UNIVERSITY



Contents

1. Overview	
1.1 Overview	4
1.2 Software Version and Tools	4
1.3 File Configuration	4
1.4 Document history	5
2. Synthesis 준비	
2.1 Synthesis 준비	7
2.2 RTL File 검증	9
2.3 SDC File 제작	11
3. Synthesis 및 PRE-STA	
3.1 Synthesis를 위한 File Load	15
3.2 Synthesis 진행 및 PRE-STA	19
3.3 Gate Level Netlist 검증	23
4. Auto P&R 준비	
4.1 Auto P&R 준비	26
4.2 Pin 위치 및 Core Size 설정	27
5. Auto P&R	
5.1 Auto P&R을 위한 File Load	30
5.2 Auto Place	34
5.3 CTS	36
5.4 Auto Route	38
5.5 Auto P&R 결과 저장	41
6. POST-STA	
6.1 Auto P&R 후 POST-STA	43
6.2 Auto P&R 후 Netlist Function 검증	46
6.3 Auto P&R 후 DRC 및 LVS	48
6.4 PEX 추출 및 Simulation 진행	50

1. Overview

1. Overview

1.1 Overview

The purpose of this paper is 0.5um Digital CMOS 2P3M 5V Design Basic tutorial.

1.2 Tools and Software Version

- Virtuoso 6.1.8 version (Cadence)
- Calibre 2023.2_16.9 (Siemens, ㄱ Mentor Graphics)
- v2lvs v2023.2_16.9 (Siemens, ㄱ Mentor Graphics)
- Design Compiler D-2010.03-SP5-2 (Synopsys)
- IC Compiler R-2020.09-SP6 (Synopsys)
- Prime Time pts_vK-2015.06-SP3-1 (Synopsys)
- Verilog Compiled Simulator H-2013.06 (Synopsys)

1.3 File Configuration

File : NSPL_0p5um_Digital_CMOS_2P3M_5V_V1p1_12T.tar.gz

- TECH
 - NSPL_0p5um_Digital_CMOS_2P3M_5V_V1p1_12T_icc2.tf : tech file (Synopsys사 양식)
 - NSPL_0p5um_Digital_CMOS_2P3M_5V_V1p1_12T_std.tf : tech file (Cadence사 양식)
 - NSPL_0p5um_Digital_CMOS_2P3M_5V_V1p1_12T_std.itf : itf file
- LIB
 - NSPL_0p5um_Digital_CMOS_2P3M_5V_V1p1_12T_std_tt.lib : liberty file
- DB
 - NSPL_0p5um_Digital_CMOS_2P3M_5V_V1p1_12T_std_tt.db : data base file
- GDS
 - NSPL_0p5um_Digital_CMOS_2P3M_5V_V1p1_12T_std.gds : gds file
- LEF
 - NSPL_0p5um_Digital_CMOS_2P3M_5V_V1p1_12T_tech.lef : library exchange format file
 - NSPL_0p5um_Digital_CMOS_2P3M_5V_V1p1_12T_std.lef : library exchange format file
- TLU
 - NSPL_0p5um_Digital_CMOS_2P3M_5V_V1p1_12T_std.tluplus : table look-up plus file
- NDM
 - NSPL_0p5um_Digital_CMOS_2P3M_5V_V1p1_12T_std_tt.ndm : new data model file
- CDL
 - NSPL_0p5um_Digital_CMOS_2P3M_5V_V1p1_12T_std.cdl : cdl file
- MAP
 - NSPL_0p5um_Digital_CMOS_2P3M_5V_V1p1_12T_icc2_label.map : icc2 label map file
 - NSPL_0p5um_Digital_CMOS_2P3M_5V_V1p1_12T_std.layermap : layer map file
 - NSPL_0p5um_Digital_CMOS_2P3M_5V_V1p1_12T_std.map : source file 용 map file
- VERILOG
 - NSPL_0p5um_Digital_CMOS_2P3M_5V_V1p1_12T_std.v : standard cell verilog
 - shift_reg_8bit.v : rtl file
 - pong_pt1.v : rtl file
 - pixel_gen.v : rtl file
 - shift_reg_8bit_tb.v : tesh bench file
 - pong_pt1_tb.v : tesh bench file
- SDC
 - shift_reg_8bit.sdc : sdc file
 - pong_pt1.sdc : sdc file

-Util	
-pin_place	: script file
-core_size_calculation	: script file
-ICC2_PIN_SETUP	: text file
-dc_cmd.tcl	: tcl file
-icc2_cmd.tcl	: tcl file
-icc2_start_cmd.tcl	: tcl file
-icc2_pin_cmd.tcl	: tcl file
-icc2_cts_cmd.tcl	: tcl file
-icc2_route_cmd.tcl	: tcl file
-icc2_fin_cmd.tcl	: tcl file
-pt_cmd.tcl	: tcl file
-PEX	
-standard cell pex(34ea)	: pex file

1.4 Document history

Rev	Date	From	Description
1.0	2026.08.18	ETRI	Added Items 1. Overview 2. Synthesis 준비 3. Synthesis 및 PRE-STA 4. Auto P&R 준비 5. Auto P&R 6. POST-STA

2. Synthesis 준비

2. Synthesis 준비

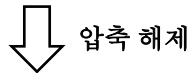
- 목적 : Synthesis 과정에서 사용하는 File 확보

1) Synthesis 준비

- Synthesis 진행을 하기 위해 File 준비하는 과정

① 제공된 Auto P&R File 압축 해제

```
NSPL_0p5um_Digital_CMOS_2P3M_5V_V1p1_12T_PnR.tar.gz
```



```
NSPL_0p5um_Digital_CMOS_2P3M_5V_V1p1_12T_PnR  
NSPL_0p5um_Digital_CMOS_2P3M_5V_V1p1_12T_PnR.tar.gz
```

② 압축 해제한 Directory Copy 및 RUN Directory 생성

```
VERILOG_pnr  
VERILOG_out  
SDC_pnr  
SDC_out  
DB  
NDM  
TLU_pnr  
GDS  
LEF  
Util  
SDF_pnr  
TLU  
CDL  
TECH  
MAP  
PEX  
VERILOG  
SDC  
LIB
```

압축 해제 후 생성된 directory copy 후 해당 directory에
RUN directory 생성 (ex. ICC2_test/RUN or ICC2_study/RUN)

1. cp -rf ICC2_REF ICC2_test



2. cd ICC2_test



3. mkdir RUN

③ Util에서 필요한 Script Copy

```
pin_place
core_size_calculation
icc2_start_cmd.tcl
icc2_route_cmd.tcl
icc2_pin_cmd.tcl
icc2_no_cts_route_cmd.tcl
icc2_fin_cmd.tcl
icc2_cts_cmd.tcl
dc_cmd.tcl
ICC2_PIN_SETUP
```

Util Directory에서 pin_place, dc_cmd.tcl File을
RUN Directory로 Copy

1. cd ICC2_test/Util



2. copy pin_place dc_cmd.tcl ../RUN

④ RTL File 확인

```
module shift_reg_8bit (
  input    clk,
  input    rst_n,
  input    din,
  output [7:0] dout
);

  reg [7:0] shift_reg;

  always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
      shift_reg <= 8'b00000000;
    end else begin
      shift_reg <= {shift_reg[6:0], din};
    end
  end

  assign dout = shift_reg;

endmodule
```

Verilog Directory에서
Synthesis 진행할 RTL File
확인 (ex. shift_reg_8bit.v)

Input, Output Pin 확인

RTL Code 확인

2) RTL File 검증

- Synthesis 진행 전 RTL File이 원하는 Function인지 확인하는 과정

① 제공된 Test Bench Sample 확인 및 수정

```
`timescale 1ns/1ps
```

```
module tb_shift_reg_8bit;
```

```
    reg        clk;
```

```
    reg        rst_n;
```

```
    reg        din;
```

```
    wire [7:0] dout;
```

```
    shift_reg_8bit uut (
```

```
        .clk    (clk),
```

```
        .rst_n  (rst_n),
```

```
        .din    (din),
```

```
        .dout   (dout)
```

```
    );
```

제공 받은 Test Bench File
(shift_reg_8bit_tb.v 중 일부)

RTL File에 기술된 Pin 이름
과 동일하게 Pin 수정

```
    always #10 clk = ~clk;
```

Clock 주기 설정

```
    initial begin
```

```
        clk    = 1'b0;
```

```
        rst_n  = 1'b0;
```

```
        din    = 1'b0;
```

일반 Signal 설정

```
        $display("=====");
```

```
        $display("  RTL-Only 8-bit Shift Register Sim  ");
```

```
        $display("=====");
```

```
        #25;
```

```
        rst_n = 1'b1;
```

```
        @(posedge clk);
```

```
        @(negedge clk);
```

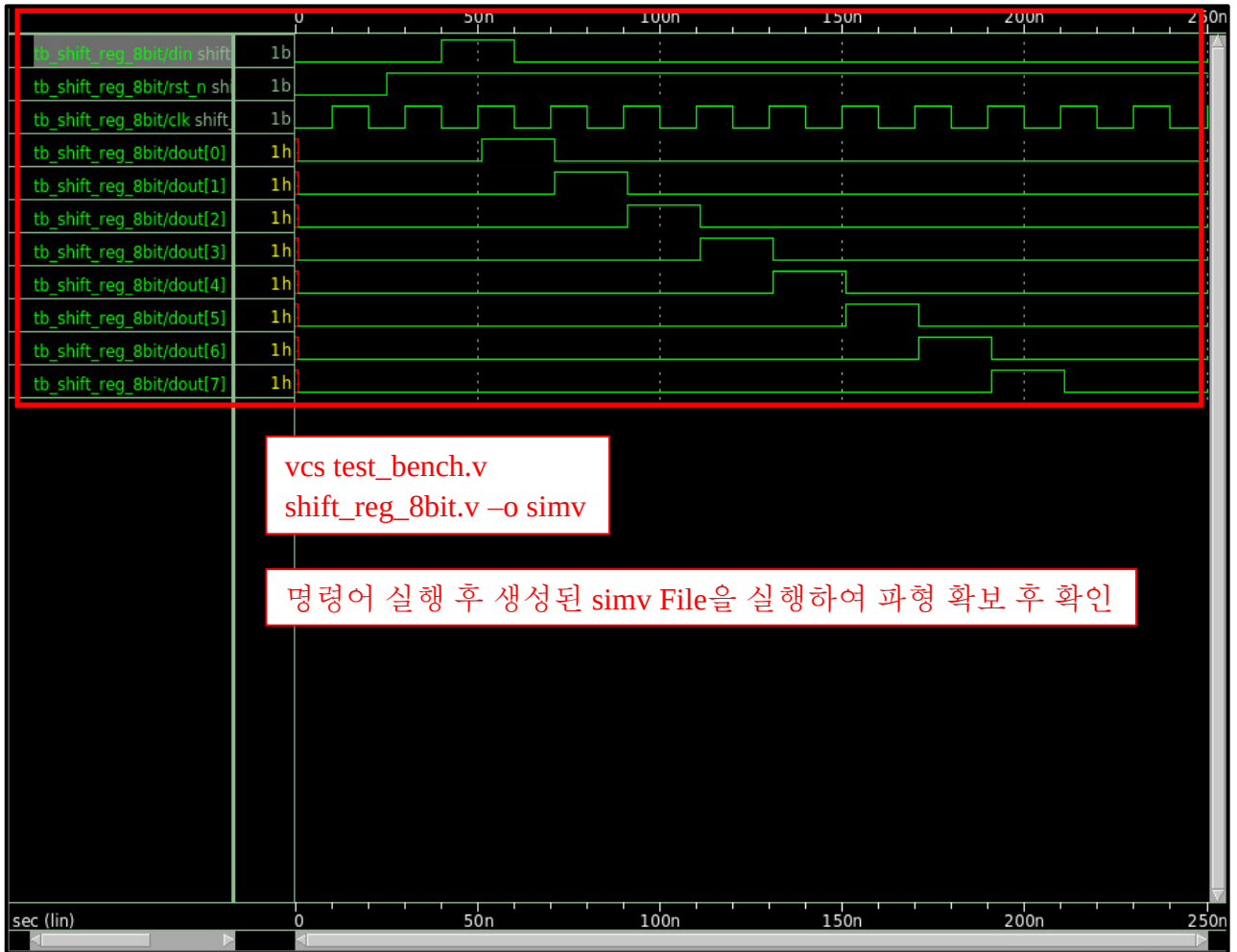
```
        din = 1'b1;
```

```
        @(posedge clk); #1;
```

```
        $display("Time=%0t ps | din=%b -> dout=%0d (Binary: %b)", $time,
```

→ 제공 받은 Test Bench File 사용 (필요 시 수정하여 사용)

② VCS(Verilog Compiled Simulator) Tool을 사용한 RTL Function Check



- Tool : VCS (Verilog Compiled Simulator)
- 설명 : Verilog 와 Test Bench Verilog File을 해석하여 DC 기반의 Simulation 진행하게 해주는 Synopsys사의 Tool (/user/tool/path /Synopsys/VCS/linux/bin/vcs)
- vcs test_bench.v rtl.v -o simv 실행 후 생성된 simv file 실행하여 Simulation 진행
- simv 실행 시 생성되는 TOP_MODULE.vcd File을 사용하여 파형 확인 가능 (ex. sx shift_reg_8bit.vcd &)
- 원하는 파형이 아닌 경우 Test Bench File 수정하여 다시 진행
- Test Bench 설정이 문제 없으면 RTL File 문제가 없는지 확인

2) SDC File 제작

- Synthesis 과정에서 사용하는 Timing의 기본 규칙서를 제작하는 과정

① 제공된 Liberty File 확인

The image shows a snippet of a Liberty File (a text-based configuration file for synthesis) with several annotations. The code is as follows:

```
cell(INVX1) {  
    cell_footprint : inv ;  
    area : 230.4 ;  
    cell_leakage_power : 0.05006 ;  
  
    pg_pin(VDD) {  
        voltage_name : VDD ;  
        pg_type : primary_power ;  
    }  
  
    pg_pin(VSS) {  
        voltage_name : VSS ;  
        pg_type : primary_ground ;  
    }  
  
    leakage_power() {  
        related_pg_pin : "VDD" ;  
        when : "A" ;  
        value : "0.05" ;  
    }  
  
    leakage_power() {  
        related_pg_pin : "VDD" ;  
        when : "!A" ;  
        value : "0.05006" ;  
    }  
  
    leakage_power() {  
        related_pg_pin : "VDD" ;  
        value : "0.05006" ;  
    }  
}
```

Annotations (pointing to the code):

- SDC File 제작 전 Liberty File 에서 Cell 별 성능 확인 (points to the top of the cell definition block)
- Cell의 기본 정보 (points to the cell_footprint, area, and cell_leakage_power lines)
- Cell의 Leakage 정보 (points to the leakage_power() blocks)

→ Cell 별 정보가 많기 때문에 필요한 정보만 빠르게 확인 (뒷장 계속)

```

pin(A) {
  direction : input ;
  related_power_pin : VDD ;
  rise_capacitance : 0.02414 ;
  rise_capacitance_range(0.02011, 0.02414);
  driver_waveform_rise : "driver_waveform_default_rise" ;
  driver_waveform_fall : "driver_waveform_default_fall" ;
  fall_capacitance : 0.02417 ;
  fall_capacitance_range(0.01914, 0.02417);
  related_ground_pin : VSS ;
  capacitance : 0.02417 ;
}

```

Cell의 Input Pin 정보

```

pin(Y) {
  direction : output ;
  related_power_pin : VDD ;
  related_ground_pin : VSS ;
  capacitance : 0 ;
  rise_capacitance : 0 ;
  fall_capacitance : 0 ;
  max_capacitance : 1.2 ;
  rise_capacitance_range(0, 0);
  fall_capacitance_range(0, 0);
  function : "(!A)" ;
}

```

Cell의 Output Pin 정보

```

timing() {
  related_pin : "A" ;
  timing_sense : negative_unate ;

  cell_fall(tmg_ntin_oload_5x5) {
    index_1("0.15, 0.35, 0.7, 1.3, 2.2");
    index_2("0.0034, 0.0133, 0.0522, 0.2043, 1.2");
    values("0.2183, 0.2346, 0.2983, 0.5478, 2.179",\
           "0.2681, 0.2867, 0.3552, 0.6017, 2.23",\
           "0.32, 0.3427, 0.4256, 0.7043, 2.32",\
           "0.3709, 0.3991, 0.4999, 0.8314, 2.487",\
           "0.4104, 0.4409, 0.5656, 0.959, 2.751");
  }
}

```

② 제공된 SDC Sample File 확인 및 수정

```
set_units -time ns -capacitance pF -resistance kOhm -voltage V -current uA

set_max_transition 2.2 [current_design]
set_max_capacitance 1.2 [current_design]
set_max_fanout 10 [current_design]

create_clock -name clk -period 20.0 [get_ports clk]

set_clock_uncertainty -setup 0.5 [get_clocks clk]
set_clock_uncertainty -hold 0.1 [get_clocks clk]

set_input_delay -max 1.0 -clock clk [get_ports {rst_n din}]
set_input_delay -min 0.1 -clock clk [get_ports {rst_n din}]

set_driving_cell -lib_cell INVX4 [get_ports {din}]

set_output_delay -max 1.1 -clock clk [get_ports {dout[7] dout[6] dout[5] do
set_output_delay -min 0.2 -clock clk [get_ports {dout[7] dout[6] dout[5] do
set_load 0.3 [get_ports {dout[7] dout[6] dout[5] dout[4] dout[3] dout[2] do
set_false_path -from [get_ports rst_n]
```

제공된 SDC Sample File
확인 및 Liberty File을 기반으로 원하는 Timing 제약 설정

- SDC (Synopsys Design Constraints) File은 Synthesis 과정에서 Timing 기준을 나타내는 규칙서 역할을 함
- 제공된 Liberty File을 기반으로 설계자가 원하는 Timing 제약을 설정하여 Synthesis에서 해당 규칙을 기반으로 진행됨
- Input Slew 및 Output Node Capacitance의 Range는 Liberty File에서 확인 가능
- 제약을 강하게 설정하면 추후 Synthesis 및 Auto P&R에서 Timing Violation 발생할 가능성이 커짐

3. Synthesis 및 PRE-STA

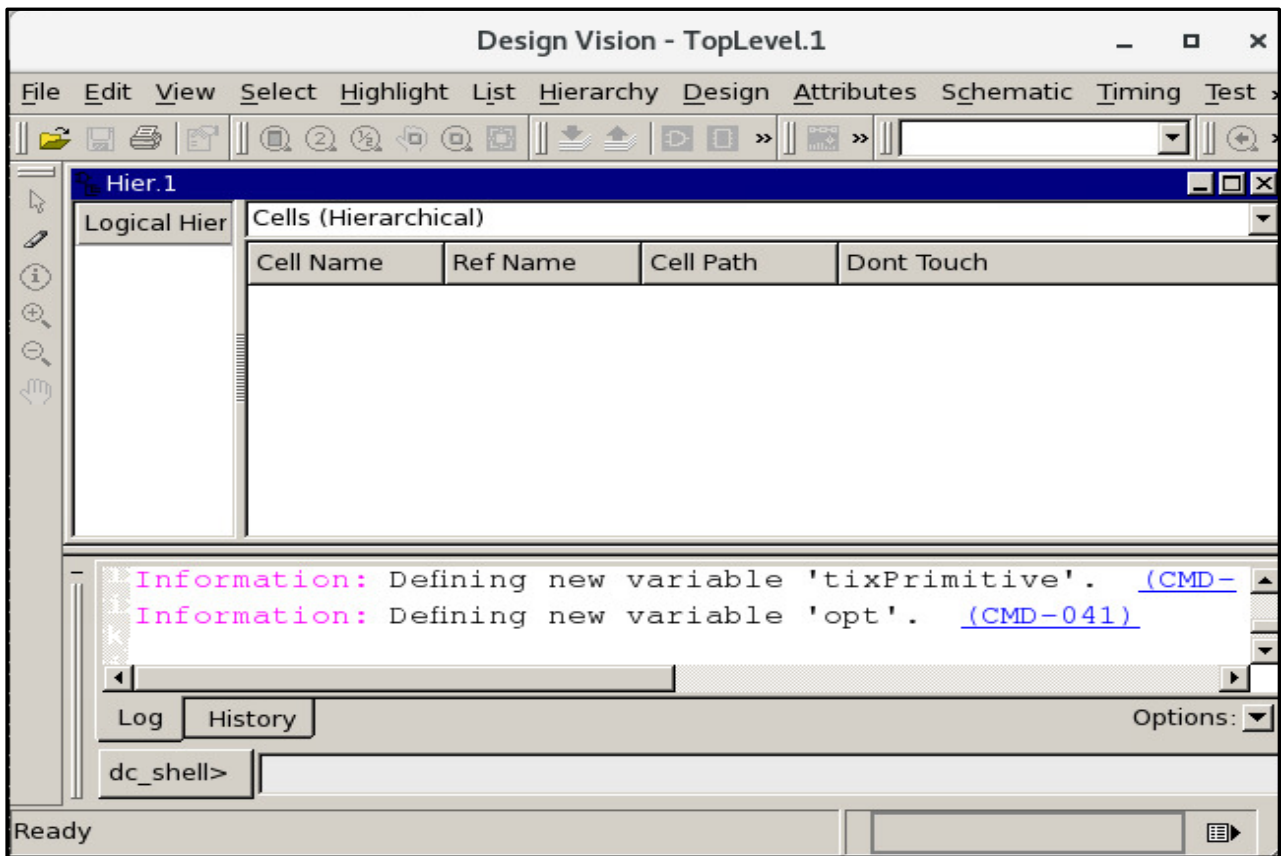
2. Synthesis 및 PRE-STA

- 목적 : Auto P&R에서 사용하는 Gate Level Netlist 확보 및 Synthesis 결과 검증

1) Synthesis를 위한 File Load

- 여러 Format으로 된 File을 Tool에 Load하여 준비하는 과정

① DC (Design Compiler) Tool 실행



```
Initializing...  
dc shell> start gui
```

- Tool : DC (Design Compiler)
- 설명 : Timing을 고려하여 RTL Netlist를 Gate Level Netlist로 변환 및 검증할 수 있는 Synopsys사의 Tool (/user/tool/path/Synopsys/DC/version/bin/dc_shell)
 - RUN Directory에서 /tool/path/dc_shell 로 DC Tool 실행
 - start_gui command를 통하여 GUI로 각 단계별 변화를 확인 가능
 - GUI로 진행하는 것이 아니라 CLI로 진행할 예정 (뒷장 계속)

② DB File 및 Verilog File Load

```
read_db ../DB/DB_FILE.db
set target_library [list ../DB/DB_FILE.db]
set link_library [list * ../DB/DB_FILE.db]
analyze -format verilog {INPUT_VERILOG}
elaborate TOP_MODULE
current_design
link
read_sdc ../SDC/TOP_MODULE.sdc
set_fix_hold
set_prefer_min [get_cip_cells -/BUR+]
compile -incr
set_change_name
set verilogout_show_unconnected_pins true
remove_attribute [get_nets -hierarchy *] dont_touch
remove_attribute [get_cells -hierarchical *] dont_touch
remove_attribute [get_nets -hierarchical *] dont_touch
remove_attribute [get_nets -hierarchical *] ideal_network
define_name_rules force lowercase -allowed "a-z0-9_" -max_length 255 -case_insensitive
change_names -rules force_lowercase -hierarchy
report_timing -delay max
report_timing -delay min
report_area > ./TOP_MODULE_area.txt
report_cell [get_cells -hierarchical -filter "is_hierarchical == false"]
report_power
report_constraint -all_violators
report_qor
write -format verilog -hierarchy -output ../VERILOG_out/TOP_MODULE.v
write_sdc ../SDC_out/TOP_MODULE.sdc
exit
```

1. DB_FILE → NSPL_0p5um_Digital_CMOS_2P3M_5V_V1p1_12T_std_tt

2. {INPUT_VERILOG} → {../VERILOG/TOP_MODULE.v}

3. TOP_MODULE → shift_reg_8bit(RTL File에 선언된 module 이름)

dc_cmd.tcl

- Synthesis 과정에서 사용하는 Standard Cell의 종류와 특성 정보가 Liberty File에 저장되어 있음
- Liberty File을 Tool이 읽을 수 있는 형식으로 바꾼 File이 DB File임
- DC Tool에서 Standard Cell을 사용할 수 있도록 DB File을 Load
- “DB_FILE”을 제공된 DB File 이름으로 변경 후 Command 실행
- Synthesis를 진행할 RTL File을 Load
- 주의 사항으로 “{INPUT_VERILOG}”에서 INPUT_VERILOG에 하위 Module의 Verilog File도 포함하여 변경해야 함
ex. {../VERILOG/top_module.v ../VERILOG/sub_module.v}
- dc_cmd.tcl 파일 및 이후 사용하는 모든 cmd.tcl File 한 번에 실행하지 말고 command 마다 개별 실행
- GUI 에서 command 실행하는 것이 아니라 CLI 창에서 실행

③ Verilog 분석 및 SDC File Load

```
read_db ../DB/DB_FILE.db
set target_library [list ../DB/DB_FILE.db]
set link_library [list * ../DB/DB_FILE.db]
analyze -format verilog {INPUT VERILOG}
elaborate TOP_MODULE
current_design TOP_MODULE
link
read_sdc ../SDC/TOP_MODULE.sdc
set_fix hold [all_clocks]
set_prefer -min [get_lib_cells */BUF*]
compile -incremental_mapping
set change_names dont_change_bus_members false
set verilogout_show_unconnected_pins true
remove_attribute [get_nets -hierarchy *] dont_touch
remove_attribute [get_cells -hierarchical *] dont_touch
remove_attribute [get_nets -hierarchical *] dont_touch
remove_attribute [get_nets -hierarchical *] ideal_network
define_name_rules force lowercase -allowed "a-z0-9_" -max_length 255 -case_insensitive
change_names -rules force_lowercase -hierarchy
report_timing -delay max
report_timing -delay min
report_area > ./TOP_MODULE_area.txt
report_cell [get_cells -hierarchical -filter "is_hierarchical == false"]
report_power
report_constraint -all_violators
report_qor
write -format verilog -hierarchy -output ../VERILOG_out/TOP_MODULE.v
write_sdc ../SDC_out/TOP_MODULE.sdc
exit
```

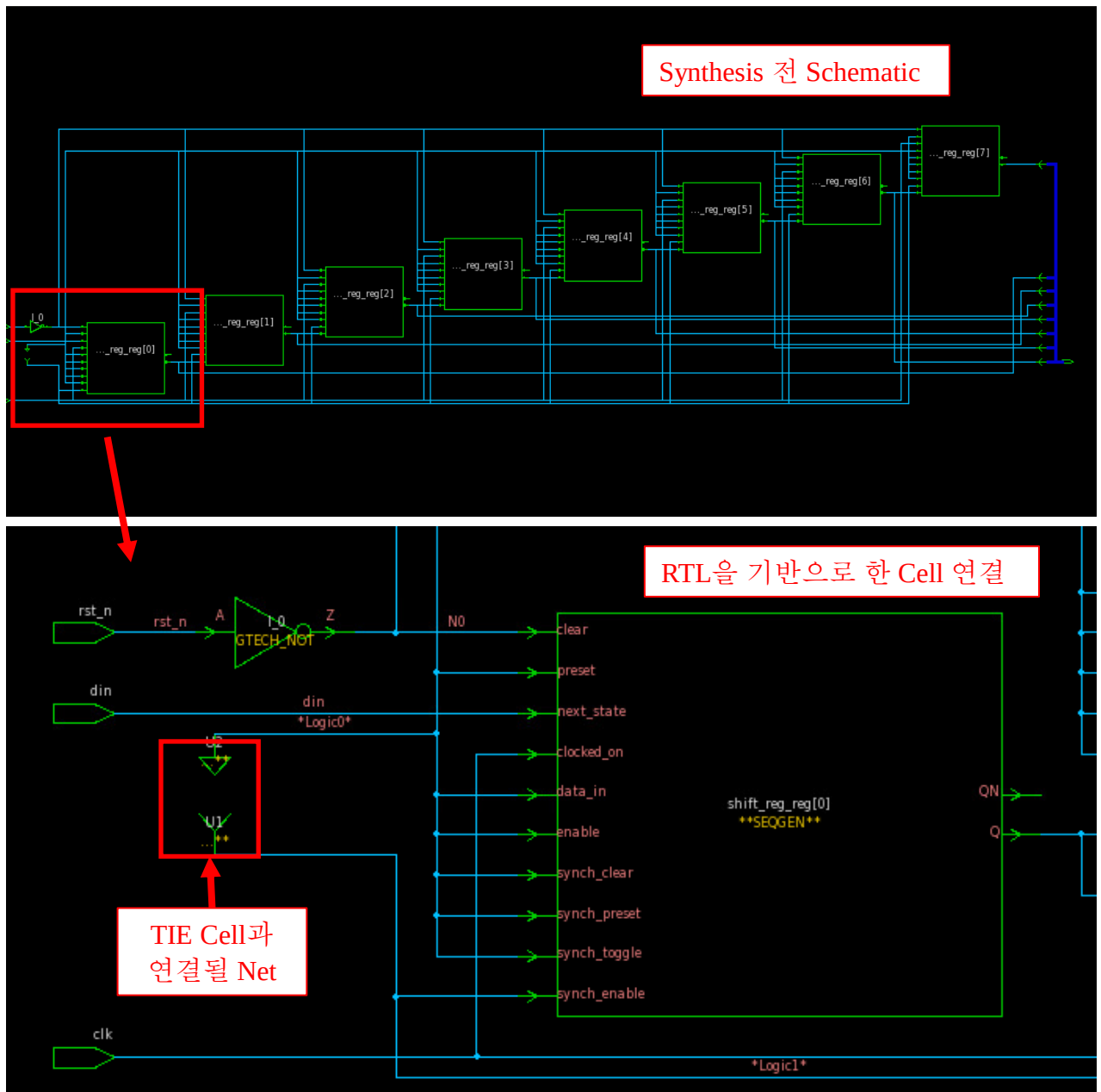
dc_cmd.tcl

```
dc_shell> elaborate shift_reg_8bit
Running PRESTO HDLC
```

```
Inferred memory devices in process
in routine shift_reg_8bit line 11 in file
'../VERILOG/shift_reg_8bit.v'.
```

Register Name	Type	Width	Bus	MB	AR	AS	SR	SS	ST
shift_reg_reg	Flip-flop	8	Y	N	Y	N	N	N	N

- “TOP_MODULE”을 상위 Module의 이름으로 변경
- Load한 RTL의 상위 Module의 이름과 DB File을 연결
- “TOP_MODULE.sdc”를 제작한 SDC File로 변경 후 Load
- CLI 환경에서 나오는 LOG 및 GUI 환경에서 나오는 Schematic 확인 (뒷장에서 계속)



- GUI 에서 Logical Hierarchy에 생긴 block을 우 클릭하여 Schematic View Open하여 확인 가능
- Synthesis를 진행하지 않았기 때문에 Standard Cell이 아닌 DC Tool이 가지고 있는 기본 Cell을 사용하여 Schematic 구현
- Synthesis 진행하면서 Standard Cell로 변경될 예정
- Cell의 Pin 중 사용하지 않는 Pin을 0 또는 1로 고정하는 방법으로 Power 직접 연결이 아닌 TIE Cell을 통한 간접 연결로 고정할 예정

2) Synthesis 진행 및 PRE-STA

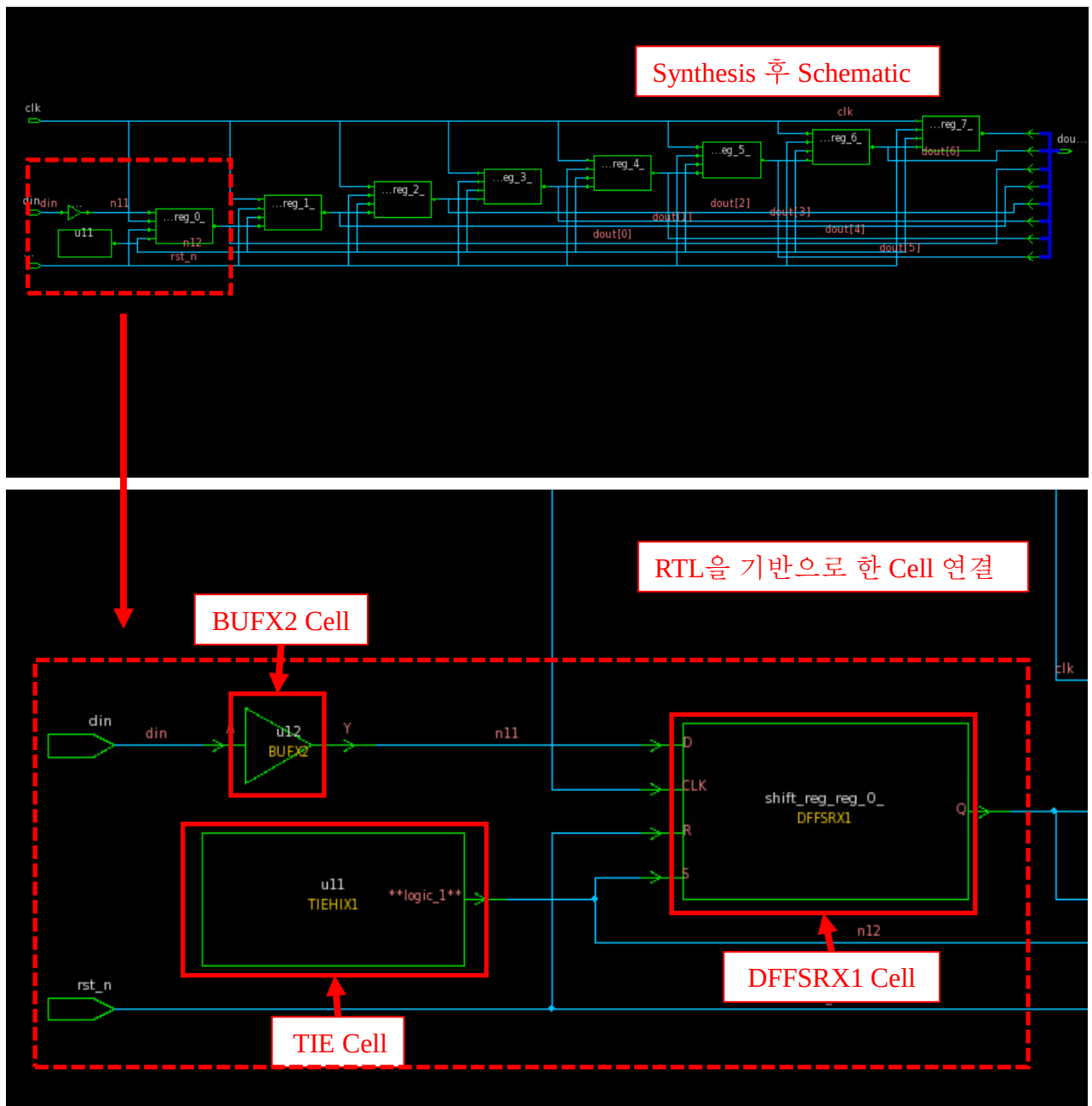
- RTL, Standard Cell, Timing 제약을 고려하여 Gate Level Netlist 확보

① Synthesis 진행

```
read_db ../DB/DB_FILE.db
set target_library [list ../DB/DB_FILE.db]
set link_library [list * ../DB/DB_FILE.db]
analyze -format verilog {INPUT_VERILOG}
elaborate TOP_MODULE
current_design TOP_MODULE
link
read_sdc ../SDC/TOP_MODULE.sdc
set_fix hold [all_clocks]
set_prefer -min [get_lib_cells */BUF*]
compile -incremental_mapping
set change_names dont_change_bus_members false
set verilogout show_unconnected_pins true
remove_attribute [get_nets -hierarchy *] dont_touch
remove_attribute [get_cells -hierarchical *] dont_touch
remove_attribute [get_nets -hierarchical *] dont_touch
remove_attribute [get_nets -hierarchical *] ideal_network
define_name_rules force_lowercase -allowed "a-z0-9_" -max_length 255 -case_insensitive
change_names -rules force_lowercase -hierarchy
report_timing -delay max
report_timing -delay min
report_area > ./TOP_MODULE.area.txt
report_cell [get_cells -hierarchical -filter "is_hierarchical == false"]
report_power
report_constraint -all_violators
report_qor
write -format verilog -hierarchy -output ../VERILOG_out/TOP_MODULE.v
write_sdc ../SDC_out/TOP_MODULE.sdc
exit
```

dc_cmd.tcl

- Timing 제약 위반시 Tool에서 해결할 수 있도록 Option 설정
- compile -incremental_mapping command를 사용하여 Synthesis 진행
- Synthesis를 통해서 DB File(Standard Cell 특성), Verilog(RTL), SDC File(Timing 제약)을 합성하여 Gate Level Netlist 확보
- 추후 Netlist를 CDL로 변환 시 대소문자가 구분되지 않아 중복 되지 않게 옵션 설정 (LVS 및 PEX 추출 시 Option에 따라 Short로 인식할 수 있음)
- remove_attribute command에서 발생하는 Error는 무시
- Synthesis 진행 후 GUI 및 CLI Log를 통해서 결과 확인 (뒷장 참고)



- Synthesis를 진행하면서 RTL 기반으로 Standard Cell을 사용하여 Schematic을 생성하기 때문에 Standard Cell과 각 Pin의 연결 확인 가능
- Timing 제약을 위반하거나 Fan Out으로 인하여 Capacitance가 커질 경우 BUF Cell을 추가하여 문제를 해결하는 것을 확인 가능
- 사용하지 않는 Pin을 TIE Cell을 통해 1 또는 0으로 고정한 것을 확인 가능

③ PRE-STA

```
read db ../DB/DB_FILE.db
set target_library [list ../DB/DB_FILE.db]
set link_library [list * ../DB/DB_FILE.db]
analyze -format verilog {INPUT_VERILOG}
elaborate TOP_MODULE
current_design TOP_MODULE
link
read sdc ../SDC/TOP_MODULE.sdc
set_fix hold [all_clocks]
set_prefer -min [get_lib cells */BUF*]
compile -incremental mapping
set change_names dont_change bus_members false
set verilogout_show_unconnected_pins true
remove_attribute [get_nets -hierarchy *] dont_touch
remove_attribute [get_cells -hierarchical *] dont_touch
remove_attribute [get_nets -hierarchical *] dont_touch
remove_attribute [get_nets -hierarchical *] ideal_network
define_name_rules force_lowercase -allowed "a-z0-9_" -max_length 255 -case_insensitive
change_names -rules force_lowercase -hierarchy
report_timing -delay max
report_timing -delay min
report_area ./TOP_MODULE.area.txt
report_cell [get_cells -hierarchical -filter "is_hierarchical == false"]
report_power
report_constraint -all_violators
report_qor
write -format verilog -hierarchy -output ../VERILOG_out/TOP_MODULE.v
write_sdc ../SDC_out/TOP_MODULE.sdc
exit
```

DC Tool Command

- Synthesis 결과가 Timing 및 Input Cap, Output Cap 등 문제가 있는지 확인
- report_timing command를 사용하여 Timing 관련 정보 확인 가능
- report_cell command를 사용하여 Synthesis 후 사용된 Cell List 확인 가능
- report_power command를 사용하여 Power 관련 정보 확인 가능
- report_constraint command를 SDC에서 작성한 제약을 기반으로 Timing을 포함하여 모든 항목에 대하여 위반 사항이 있는지 확인 가능
- report_qor command를 사용하여 종합적인 정보 확인 가능
- report_area command를 사용하여 Gate Level Netlist에서 사용한 모든 Standard Cell의 면적 정보를 확인 가능 (Auto P&R에서 Core Size 설정에서 해당 report 결과물 사용할 예정으로 따로 저장
ex. > shift_reg_8bit_area.txt)
- 문제가 없으면 write command를 통하여 저장하고 문제가 있으면 SDC 수정하여 Synthesis 처음부터 다시 진행 (뒷장에서 계속)

Point	Incr	Path

clock clk (rise edge)	0.00	0.00
clock network delay (ideal)	0.00	0.00
shift_reg_reg_0 /CLK (DFFSRX1)	0.00	0.00 r
shift_reg_reg_0 /Q (DFFSRX1)	2.40	2.40 f
dout[0] (out)	0.00	2.40 f
data arrival time		2.40

clock clk (rise edge)	20.00	20.00
clock network delay (ideal)		
clock uncertainty		
output external delay		
data required time		13.50

data required time		13.50
data arrival time		-2.40

slack (MET)		11.10

Slack 값이 양수가 나오면 Pass
음수가 나오면 Fail

Timing 검증 (Max)

Point	Incr	Path

clock clk (rise edge)	0.00	0.00
clock network delay (ideal)	0.00	0.00
input external delay	0.50	0.50 f
din (in)	0.01	0.51 f
shift_reg_reg_0 /D (DFFSRX1)	0.00	0.51 f
data arrival time		0.51

clock clk (rise edge)	0.00	0.00
clock network delay (ideal)	0.00	0.00
clock uncertainty		
shift_reg_reg_0 /CLK (DFFSRX1)		
library hold time		
data required time		0.13

data required time		0.13
data arrival time		-0.51

slack (MET)		0.38

Slack 값이 양수가 나오면 Pass
음수가 나오면 Fail

Timing 검증 (Min)

This design has no violated constraints.

Constraint 검증 (Violators)

- report command 결과가 다음과 같이 양수 또는 no violated인 것을 확인
- 문제가 없으면 DC Tool 종료

3) Gate Level Netlist 검증

- 확보한 Gate Level Netlist가 원하는 Function을 수행하는지 검증하는 과정

① Test Bench 수정

```
`timescale 1ns/1ps

module tb_shift_reg_8bit;

    reg        clk;
    reg        rst_n;
    reg        din;
    wire [7:0] dout;

    shift_reg_8bit uut (
        .clk_   (clk),
        .rst_n_ (rst_n),
        .din_   (din),
        .dout_  (dout)
    );

    always #10 clk = ~clk;

initial begin
    clk  = 1'b0;
    rst_n = 1'b0;
    din  = 1'b0;

    $display("=====");
    $display("  RTL-Only 8-bit Shift Register Sim  ");
    $display("=====");

    #25;
    rst_n = 1'b1;
    @(posedge clk);

    @(negedge clk);
    din = 1'b1;
    @(posedge clk); #1;
    $display("Time=%0t ps | din=%b -> dout=%0d (Binary: %b)", $time,
```

제공 받은 Test Bench File
(shift_reg_8bit_tb.v 중 일부)

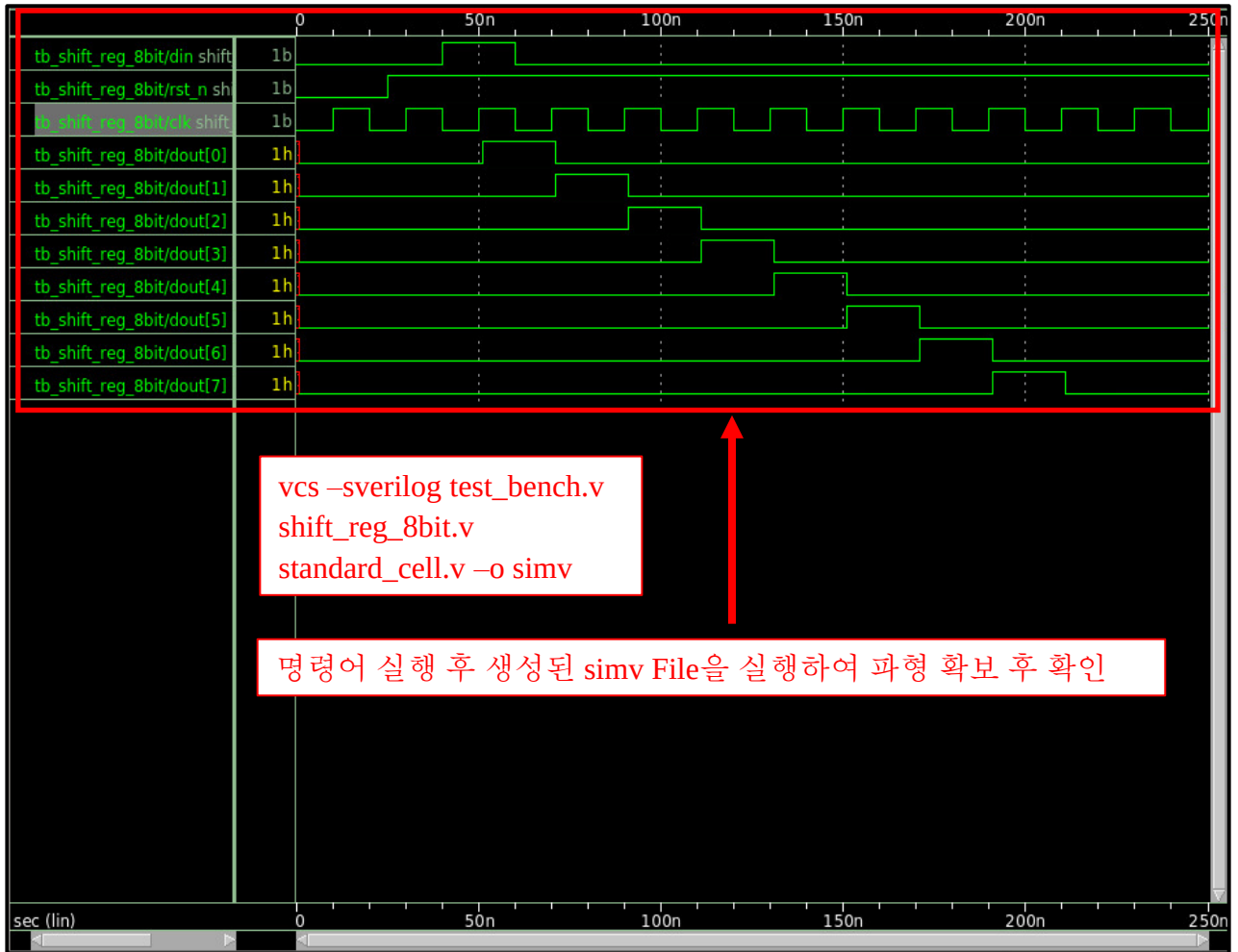
Input, Output Pin 설정

Clock 주기 설정

Clock에 따른 Signal 설정

→ RTL 검증에서 사용한 Test Bench File 사용 (필요 시 수정하여 사용)

② VCS Tool을 사용한 Gate Level Netlist Function Check



- 제공 받은 Standard Cell Function이 기술된 Verilog도 포함되어 진행
- 원하는 파형이 아닌 경우 Test Bench File 수정하여 다시 진행
- Tesh Bench 설정이 문제 없으면 Timing 제약 완화하여 다시 진행

4. Auto P&R 준비

4. Auto P&R 준비

- 목적 : Auto P&R 과정에서 필요한 File 확보

1) Auto P&R 준비

- Auto P&R 진행하기 위해서 준비하는 과정

① 필요한 Script 준비

```
pin_place
core_size_calculation
icc2_start_cmd.tcl
icc2_route_cmd.tcl
icc2_pin_cmd.tcl
icc2_no_cts_route_cmd.tcl
icc2_fin_cmd.tcl
icc2_cts_cmd.tcl
dc_cmd.tcl
ICC2_PIN_SETUP
```

Util Directory에서 dc_cmd.tcl File을 제외한
모든 File RUN Directory로 Copy

1. cd ICC2_test/Util

2. copy `ls -d \* | grep -v dc` ../RUN

② Auto P&R 진행 과정에서 필요한 File이 있는지 확인

```
VERILOG_pnr
VERILOG_out ← Synthesis에서 Write한 .v File
SDC_pnr
SDC_out ← Synthesis에서 Write한 .sdc File
DB
NDM ← NSPL_0p5um_Digital_CMOS_2P3M_5V_V1p1_12T_std_tt.ndm
TLU_pnr
GDS ← NSPL_0p5um_Digital_CMOS_2P3M_5V_V1p1_12T_std.gds
LEF
Util
SDF_pnr
TLU ← NSPL_0p5um_Digital_CMOS_2P3M_5V_V1p1_12T_std.tluplus
CDL
TECH ← NSPL_0p5um_Digital_CMOS_2P3M_5V_V1p1_12T_icc2.tf
MAP ← NSPL_0p5um_Digital_CMOS_2P3M_5V_V1p1_12T_icc2_label.map
PEX
VERILOG
SDC
LIB
```

2) Pin 위치 및 Core Size 설정

- Auto P&R에서 사용할 Pin 위치 및 Core Size 결정하는 과정

① Place Pin 위치 설정

```
# === PIN SET UP === #
#      28 27 26 25 24 23 22
#  1
#  2
#  3
#  4
#  5
#  6
#  7
#      8  9 10 11 12 13 14

# LEFT
PIN1 = clk
PIN2 =
PIN3 =
PIN4 = # VSS
PIN5 =
PIN6 = din
PIN7 =

# BOTTOM
PIN8 = # VDD
PIN9 = # VSS
```

Auto P&R에서
Standard Cell
Place 및 Routing 되는
Core 영역

원하는 Pin 위치에 Pin 이름 기입

Power에 경우 빈칸 또는 주석으로 기입

사용하지 않는 Pin은 빈칸으로 처리

ICC2_PIN_SETUP File 중 일부

- ICC2_PIN_SETUP File에서 PAD를 고려한 Pin 위치 설정
- 제공된 File의 양식에 맞춰 File 수정(양식에 맞춰 Script 제작되어 있음)
- Power가 들어올 Pin은 빈칸 또는 주석으로 기입
- Bus 형식 Pin은 Pin[0], Pin[1]과 같이 개별 Pin으로 기입
- 제공된 pin_place script를 활용하여 Auto P&R에서 사용하는 pin_setup_file 제작 예정 (core_size 정한 이후 진행)

② Core Size 설정

```
#!/bin/bash
DC_SYNTHESIS_REPORT_AREA=`cat ./TOP_MODULE_area.txt | grep -e
CORE_SIZE_X='150'
CORE_SIZE_Y='150'

echo -n 'Core Utilization : '
echo $CORE_SIZE_X $CORE_SIZE_Y $DC_SYNTHESIS_REPORT_AREA | awk
echo -n 'Core CPP      : '
echo $CORE_SIZE_X $CORE_SIZE_Y $DC_SYNTHESIS_REPORT_AREA | awk
echo -n 'Core Height : '
echo $CORE_SIZE_X $CORE_SIZE_Y
```

제공된 core_size_calculation script 중 일부



실행 결과

```
Core Utilization : 83.6957%
Core CPP      : 147.2[um] | 46[cpp]
Core Height : 144[um] | 4[row]
```

Script 실행 결과

- “TOP_MODULE_area.txt”를 Synthesis 과정에서 report_area log를 저장한 File 이름으로 변경
- Core Utilization은 Core 면적에서 Standard Cell의 면적 비율을 의미함
- Core Utilization이 크면 Standard Cell 배치 및 배선에서 문제가 발생할 확률이 증가함
- 일반적인 Auto P&R에서는 매우 많은 Standard Cell을 사용하고 Core 면적도 매우 크게 하여 Core Utilization을 85% 이상으로 설정함
- Standard Cell을 적게 사용하는 경우 Core Utilization을 매우 적게 설정해서 진행하거나 Timing 고려하는 것을 일부 타협해야 함
- Standard Cell을 적게 사용하고 Core Utilization을 크게 설정해서 진행하는 경우에 Auto Place와 POST-STA에서 문제가 발생할 확률이 매우 큼
- Core Utilization을 고려하여 X와 Y 값을 변경하며 Core Size 결정
- Core Size가 정해지면 pin_place script를 사용하여 Core Size에 맞는 Pin 위치 정보를 포함한 pin_setup_file 생성

5. Auto P&R

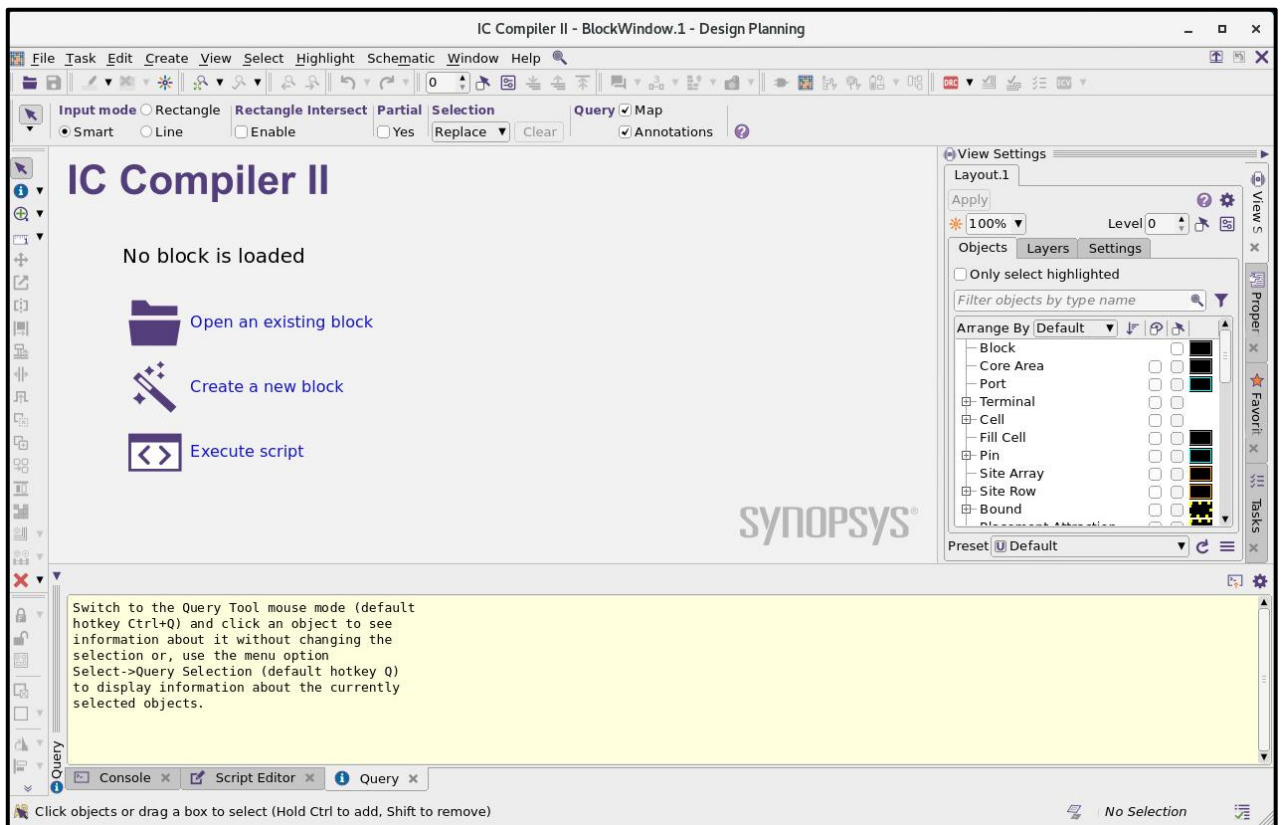
5. Auto P&R

- 목적 : Auto Place and Routing 진행

1) Auto P&R을 위한 File Load

- 여러 Format으로 된 File을 Tool에 Load하여 준비하는 과정

① ICC2(IC Compiler2) Tool 실행



```
icc2 shell> start_gui
```

- Tool : ICC2 (IC Compiler2)
- 설명 : Auto Place and Routing 진행하는 Synopsys사의 Tool (/user/tool/path/Synopsys/ICC/version/bin/icc2_shell)

- RUN Directory에서 /tool/path/icc2_shell command로 ICC2 Tool 실행
- start_gui command를 통하여 GUI로 각 단계별 변화를 확인 가능
- GUI로 진행하는 것이 아니라 CLI로 진행할 예정 (뒷장 계속)

② Auto P&R에서 필요한 File Load

```
create_lib -technology ../TECH/NSPL_0p5um_Digital_CMOS_2P3M_5V_V1p1_12T_1cc2.tf -ref_lib
create_block TOP MODULE start
read_verilog ../VERILOG_out/TOP_MODULE.v
create_corner CORNER
current_corner CORNER
read_parasitic_tech -tlup ../TLU/TLU_FILE.tluplus
set_parasitic_parameters -early_spec ../TLU/TLU_FILE.tluplus -late_spec ../TLU/TLU_FILE
create_scenario -corner CORNER -name scenario_CORNER
current_scenario scenario_CORNER
read_sdc -version 1.8 ../SDC_out/TOP_MODULE.sdc
set_app_options -name place.coarse.continue_on_missing_scandef -value true
create_net -power VDD
create_net -ground VSS
initialize_floorplan -control_type core -shape R -side_length {CORE_SIZE_X CORE_SIZE_Y}
remove_tracks -all
create_track -layer MET1 -coord 1.5 -dir Y -bbox {{0 0} {CORE_SIZE_X CORE_SIZE_Y}} -spac
create_track -layer MET2 -coord 1.6 -dir X -bbox {{0 0} {CORE_SIZE_X CORE_SIZE_Y}} -spac
create_track -layer MET3 -coord 3.0 -dir Y -bbox {{0 0} {CORE_SIZE_X CORE_SIZE_Y}} -spac
create_pg_region pg_region1 -core -expand by_edge {{{side: left}{offset: 0}}} {{side: top
create_pg_std_cell_conn_pattern std PGrail -layers {MET1} -rail_width {2.0}
set_pg_strategy S1 -core -pattern {{name: std_PGrail}{nets: {VDD VSS}}}
compile_pg
create_pg_mesh_pattern mesh_pattern_VDD -layers {{{vertical_layer: MET2} {width: 2.0} {p
set_pg_strategy VDD mesh -pattern {{name: mesh_pattern_VDD} {nets: VDD}} -pg_regions pg
compile_pg -strategies VDD mesh
create_pg_mesh_pattern mesh_pattern_VSS -layers {{{vertical_layer: MET2} {width: 2.0} {p
set_pg_strategy VSS mesh -pattern {{name: mesh_pattern_VSS} {nets: VSS}} -pg_regions pg
compile_pg -strategies VSS mesh
set_app_options -list {place.legalize.avoid_pins_under_preroute_layers {MET2}}
set GetLibPins [get lib_pins]
set_app_options -list [list place.legalize.avoid_pins_under_preroute_libpins [get object
set_app_options -list {plan.place.congestion_driven_mode both}
set_app_options -name opt.tie_cell.max_fanout -value 4
report_cells [get_cells -hierarchical -filter "is_hierarchical"]
close_blocks -save
```

ICC2 Tool Command
(icc2_cmd.tcl 중 일부)

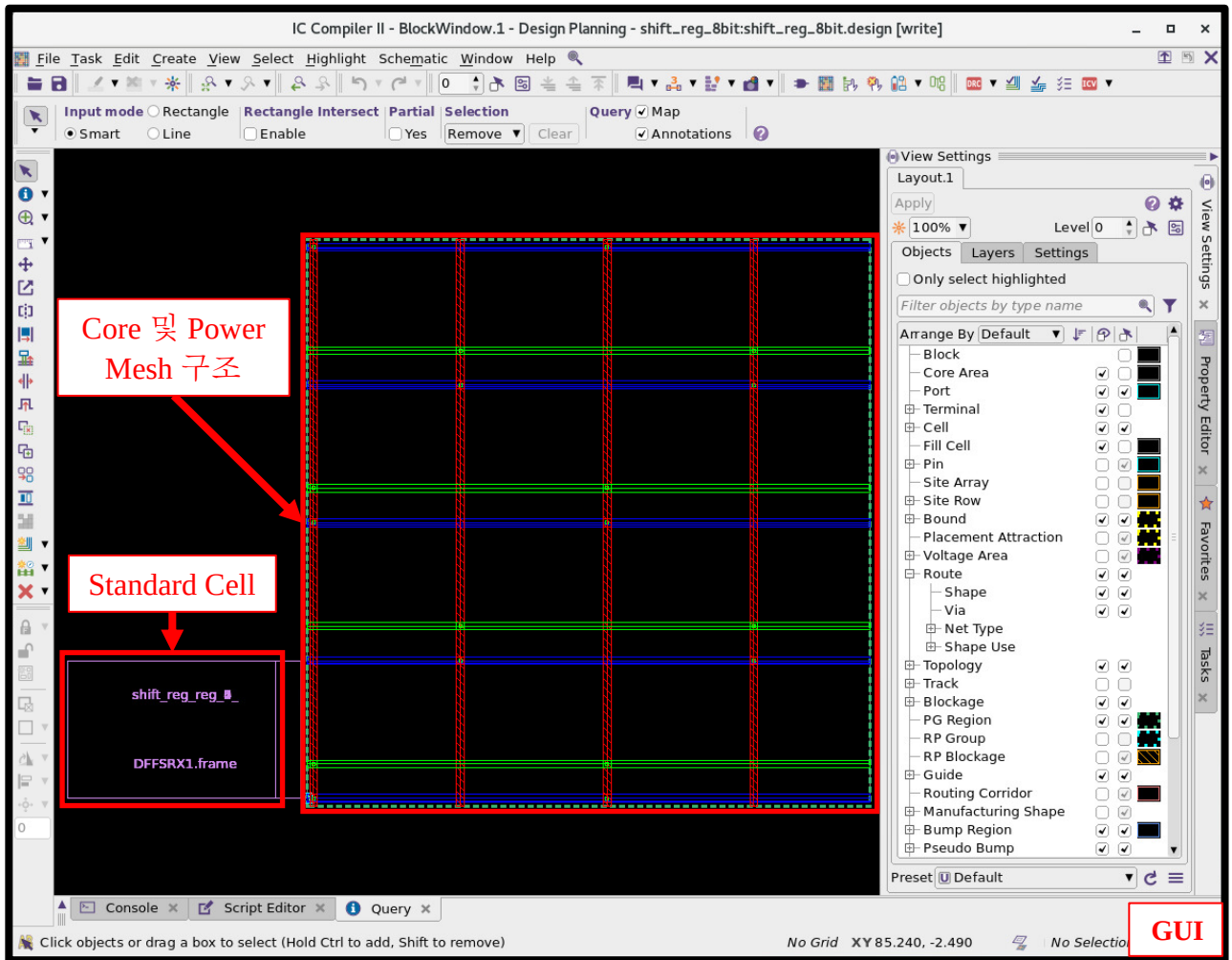
- NDM, VERILOG, SDC, TOP_MODULE 등 이전과 동일하게 변경
(Verilog와 SDC는 Synthesis한 결과물을 사용하고 TLU, TECH 등 다른
File은 제공된 File 사용)
- Auto P&R을 진행할 Library 생성 (Tech File, NDM Load)
- Auto P&R을 진행할 Block 생성 (Layout 작업 시 Library 내부 Cell과 유사)
- Auto P&R을 진행할 Gate Level Netlist Load
- Auto P&R에서 참고하는 TLU File Load
- Auto P&R에서 참고하는 SDC File Load
- Close_blocks -save command 사용 이유는 Auto Place 단계 참고

③ Tool Setup

```
create_lib -technology ../TECH/NSPL_0p5um_Digital_CMOS_2P3M_5V_V1p1_12T_icc2.tf -ref_lib
create_block TOP MODULE start
read_verilog ../VERILOG_out/TOP_MODULE.v
create_corner CORNER
current_corner CORNER
read_parasitic_tech -tlup ../TLU/TLU_FILE.tluplus
set_parasitic_parameters -early_spec ../TLU/TLU_FILE.tluplus -late_spec ../TLU/TLU_FILE
create_scenario -corner CORNER -name scenario_CORNER
current_scenario scenario_CORNER
read_sdc -version 1.8 ../SDC_out/TOP_MODULE.sdc
set_app_options -name place.coarse.continue_on_missing_scandef -value true
create_net -power VDD
create_net -ground VSS
initialize_floorplan -control_type core -shape R -side_length {CORE_SIZE_X CORE_SIZE_Y}
remove_tracks -all
create_track -layer MET1 -coord 1.5 -dir Y -bbox {{0 0} {CORE_SIZE_X CORE_SIZE_Y}} -spac
create_track -layer MET2 -coord 1.6 -dir X -bbox {{0 0} {CORE_SIZE_X CORE_SIZE_Y}} -spac
create_track -layer MET3 -coord 3.0 -dir Y -bbox {{0 0} {CORE_SIZE_X CORE_SIZE_Y}} -spac
create_pg_region pg_region1 -core -expand_by_edge {{{side: left}{offset: 0}} {{side: top
create_pg_std_cell_conn_pattern std PGrail -layers {MET1} -rail_width {2.0}
set_pg_strategy S1 -core -pattern {{name: std_PGrail}{nets: {VDD VSS}}}
compile_pg
create_pg_mesh_pattern mesh_pattern_VDD -layers {{{vertical_layer: MET2} {width: 2.0} {p
set_pg_strategy VDD mesh -pattern {{name: mesh_pattern_VDD} {nets: VDD}} -pg_regions pg
compile_pg -strategies VDD mesh
create_pg_mesh_pattern mesh_pattern_VSS -layers {{{vertical_layer: MET2} {width: 2.0} {p
set_pg_strategy VSS mesh -pattern {{name: mesh_pattern_VSS} {nets: VSS}} -pg_regions pg
compile_pg -strategies VSS mesh
set_app_options -list {place.legalize.avoid_pins_under_preroute_layers {MET2}}
set GetLibPins [get lib_pins]
set_app_options -list [list place.legalize.avoid_pins_under_preroute_libpins [get object
set_app_options -list {plan.place.congestion_driven_mode both}
set_app_options -name opt.tie_cell.max_fanout -value 4
report_cells [get_cells -hierarchical -filter "is_hierarchical"]
close_blocks -save
```

ICC2 Tool Command
(icc2_cmd.tcl 중 일부)

- Auto P&R을 진행하면서 사용하는 ICC2 Tool Option Setup 및 Core 영역 Size 및 Power Mesh 구조, Metal Track 등 설정
- “CORE_SIZE_X”와 “CORE_SIZE_Y”를 이전에 설정했던 값으로 변경
- initialize_floorplan command로 Core 영역 생성
- create_track command로 Auto Routing에서 사용하는 Metal Track 생성
- pg 관련 command로 Power Mesh 구조 생성
- 다른 command는 Auto P&R 진행 과정에서 발생할 수 있는 문제를 미리 방지하기 위해서 설정하는 Option으로 제공된 command 수정 없이 사용 (뒷장에서 계속)



```
icc_shell> initialize_floorplan -control_type core -shape R -side_length {150 150} -core_utilization false -origin offset {0 0}
Creating core...
Core utilization ratio = 83.70%
Unplacing all cells...
Creating site rows...
Creating routing tracks...
Initializing floorplan completed.
```

Core Utilization

Core Size

CLI Log

- GUI에서 Core 영역과 Power Mesh 구조 확인 가능
- Auto Place 단계에서 배치될 Standard Cell 확인 가능
- CLI Log에서 입력한 Command의 결과를 확인 가능

2) Auto Place

- Auto P&R 중 Place 및 CTS 준비 과정

① Pin Place 및 Standard Cell Auto Place

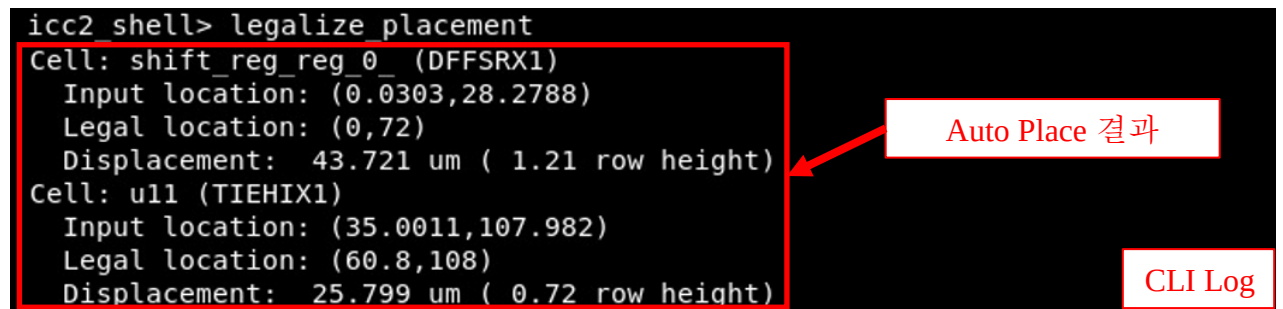
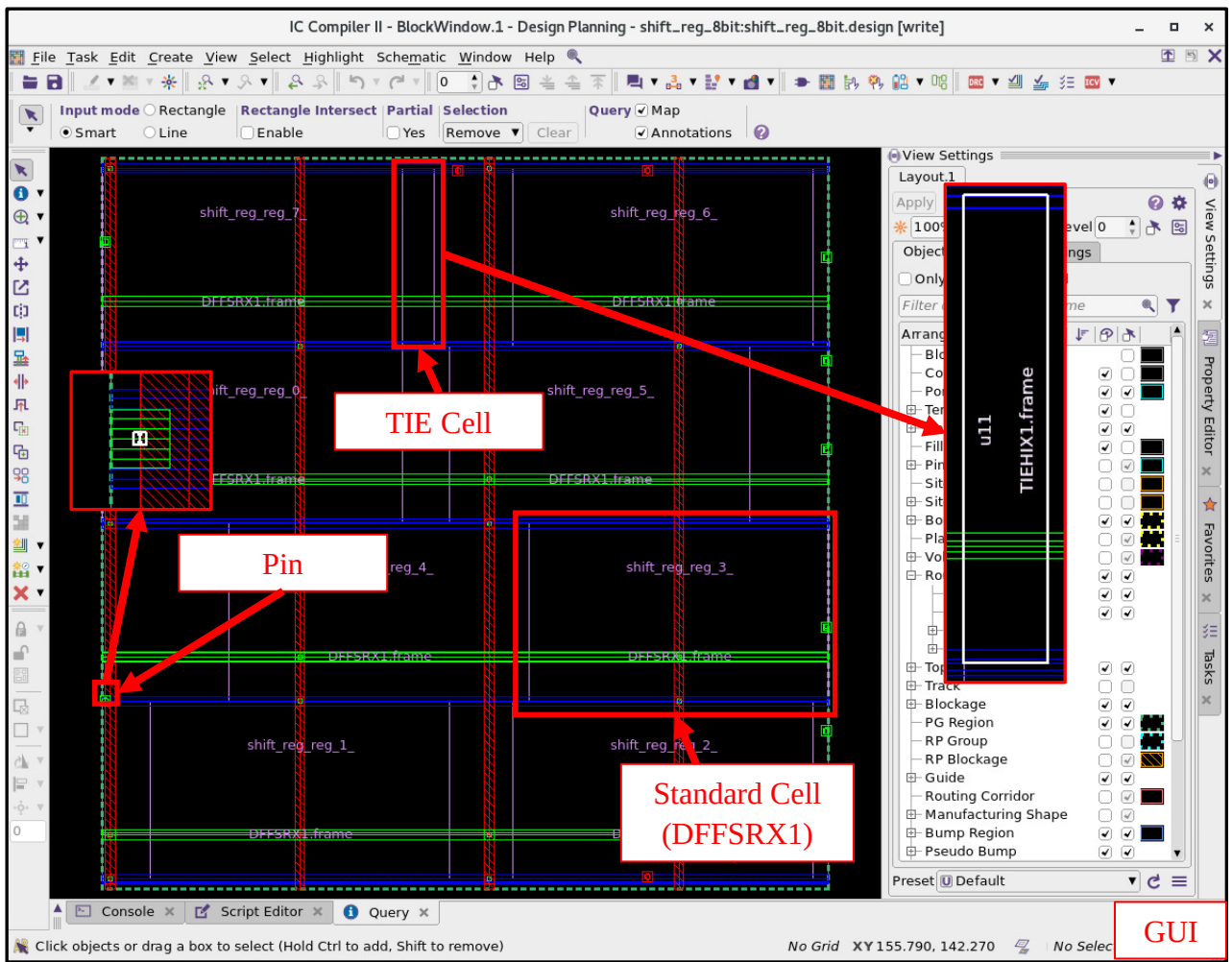
```
open_lib TOP_MODULE
copy_block -force -from_block TOP_MODULE_start -to_block TOP_MODULE_pin
open_block TOP_MODULE_pin
remove_individual_pin_constraints
read_pin_constraints -file_name pin_setup_file
place_pins -ports *
save_block -force
create_placement -floorplan
legalize_placement
close_blocks -save
```

ICC2 Tool Command
(icc2_cmd.tcl 중 일부)

```
start physical pin constraints;
{net clk} {sides 1} {offset 131.25} {layers MET3};
{net din} {sides 1} {offset 37.5} {layers MET3};
{net dout[7]} {sides 2} {offset 112.5} {layers MET2};
{net rst_n} {sides 2} {offset 75} {layers MET2};
{net dout[1]} {sides 3} {offset 131.25} {layers MET3};
{net dout[2]} {sides 3} {offset 112.5} {layers MET3};
{net dout[3]} {sides 3} {offset 93.75} {layers MET3};
{net dout[4]} {sides 3} {offset 56.25} {layers MET3};
{net dout[5]} {sides 3} {offset 37.5} {layers MET3};
{net dout[6]} {sides 3} {offset 18.75} {layers MET3};
{net dout[0]} {sides 4} {offset 37.5} {layers MET2};
end physical pin constraints;
```

pin_setup_file

- 이전과 동일하게 TOP_MODULE 이름 변경
- 이전에 진행한 Block을 copy하여 Auto Place 진행할 Block 생성
- 이전에 Script로 생성한 pin_setup_file을 사용하여 Pin Place
- Standard Cell은 Pin과 다르게 Auto Place 진행
- Netlist 기반으로 Timing 고려하지 않고 연결만 고려하여 Auto Place 됨
- 처음 생성한 Block을 계속 사용하지 않고 open_block 및 close_blocks -save command를 활용하여 Check Point 생성
(다시 진행하는 경우에 처음부터 진행하는 것이 아니라 필요한 Block에서 다시 진행 가능하기 때문에 해당 과정 필요)



- Auto Place 된 Standard Cell 확인 가능
- Cell Size가 작은 경우 Cell 이름 확인 불가 확대하여 확인 가능
- ICC2_PIN_SETUP File에서 설정한 Pin 위치를 기반으로 Pin Place 확인 가능

3) CTS

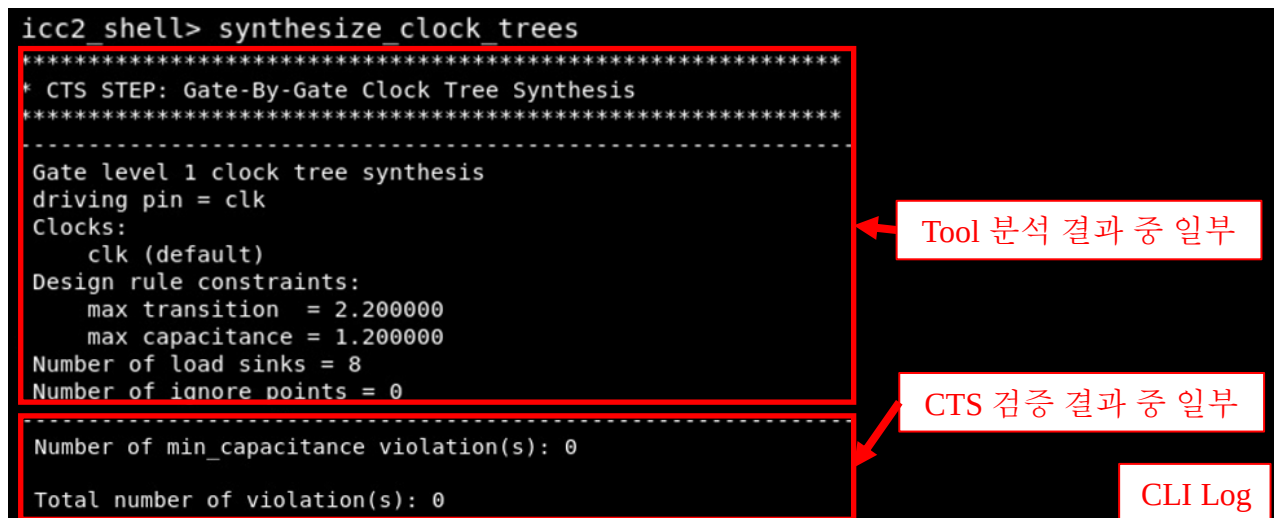
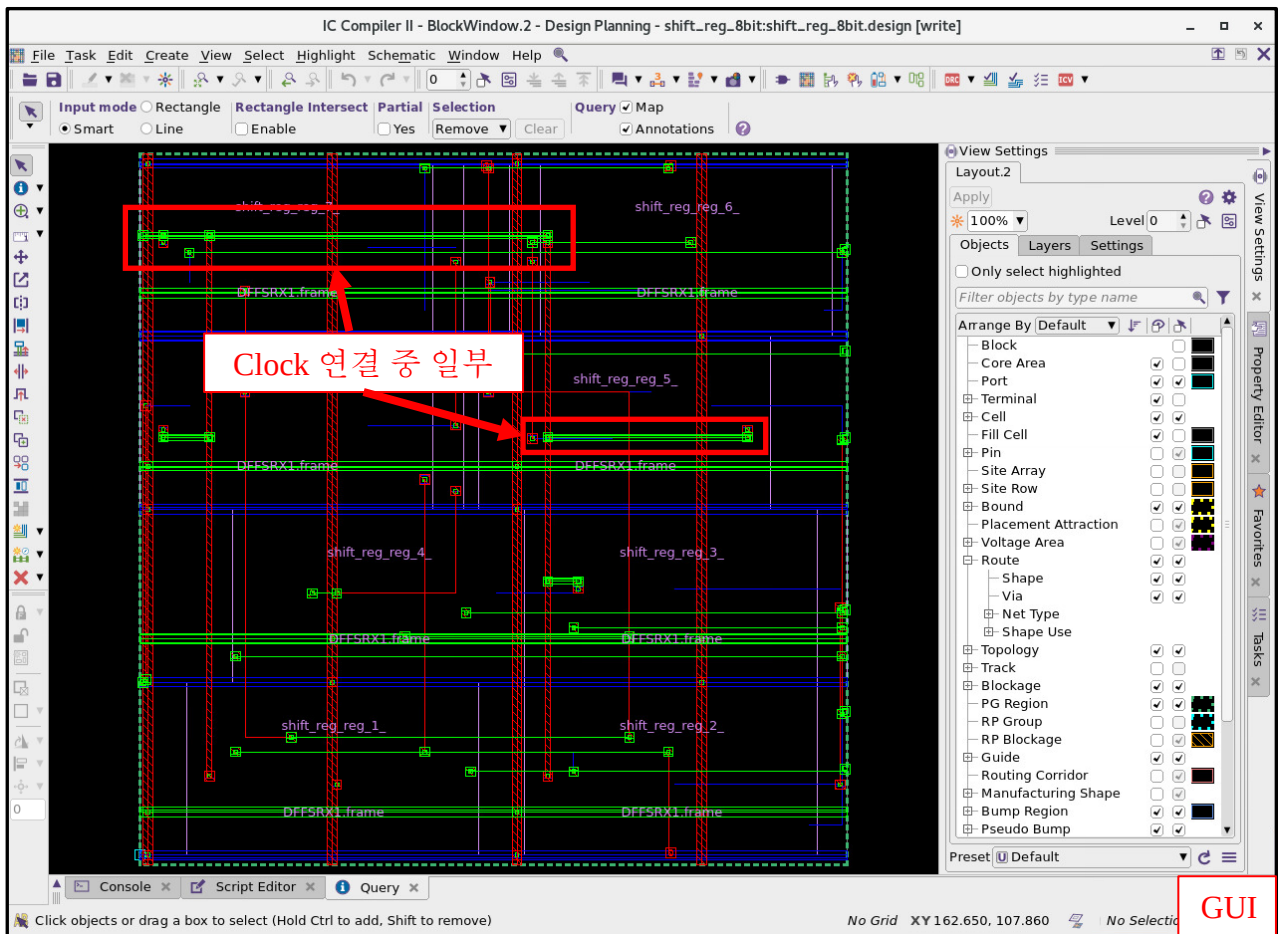
- Clock Tree 생성하는 과정

① CTS (Clock Tree Synthesis)

```
open_lib TOP_MODULE
copy_block -force -from_block TOP_MODULE_pin -to_block TOP_MODULE_cts
open_block TOP_MODULE_cts
update_timing
set_clock_tree_reference_subset -clocks [get_clocks] -lib_cells { CLKBU
check_clock_trees
check_legality
synthesize_clock_trees
legalize_placement
place_opt
update_timing
report_constraints -all_violators
clock_opt
update_timing
report_constraints -all_violators
report_cells [get_cells -hierarchical -filter "is_hierarchical == false
close_blocks -save
```

ICC2 Tool Command (icc2_cmd.tcl 중 일부)

- Clock 신호에 문제가 있는 경우에는 FF Cell 같은 경우 이상한 값을 저장하여 출력하는 문제가 발생함
- CTS(Clock Tree Synthesis)를 진행하여 문제를 사전에 방지함
- CTS 과정에서 Tool이 Clock의 Timing, Fan Out, Cell 별 Clock 도달 시간 등 여러 항목을 고려하여 BUF 추가, Cell 위치 변경과 같은 다양한 방법으로 해결함
- 이전에 진행한 Block을 copy하여 CTS 진행할 Block 생성
- Auto Place된 Cell들의 Timing 정보를 Update 진행 후 Clock Tree 확인 및 Standard Cell 규격에 맞게 배치 되었는지 확인
- Clock Pin과 Clock을 사용한 Cell의 연결을 Tool에서 분석
- Tool이 분석한 결과를 기반으로 Cell 배치 변경 및 BUF 추가
- report command를 사용하여 CTS 결과 검증 (문제가 있을 경우 Timing 제약 완화하여 Synthesis 단계부터 다시 진행)
- GUI와 CLI Log로 자세한 정보 확인 가능 (뒷장에서 계속)



- GUI에서 Clock의 배선 Point를 확인 가능
- CLI Log를 통해 Tool이 분석한 결과 확인 가능
- CLI Log를 통해 CTS 결과 검증 가능 (report command 사용)

4) Auto Route

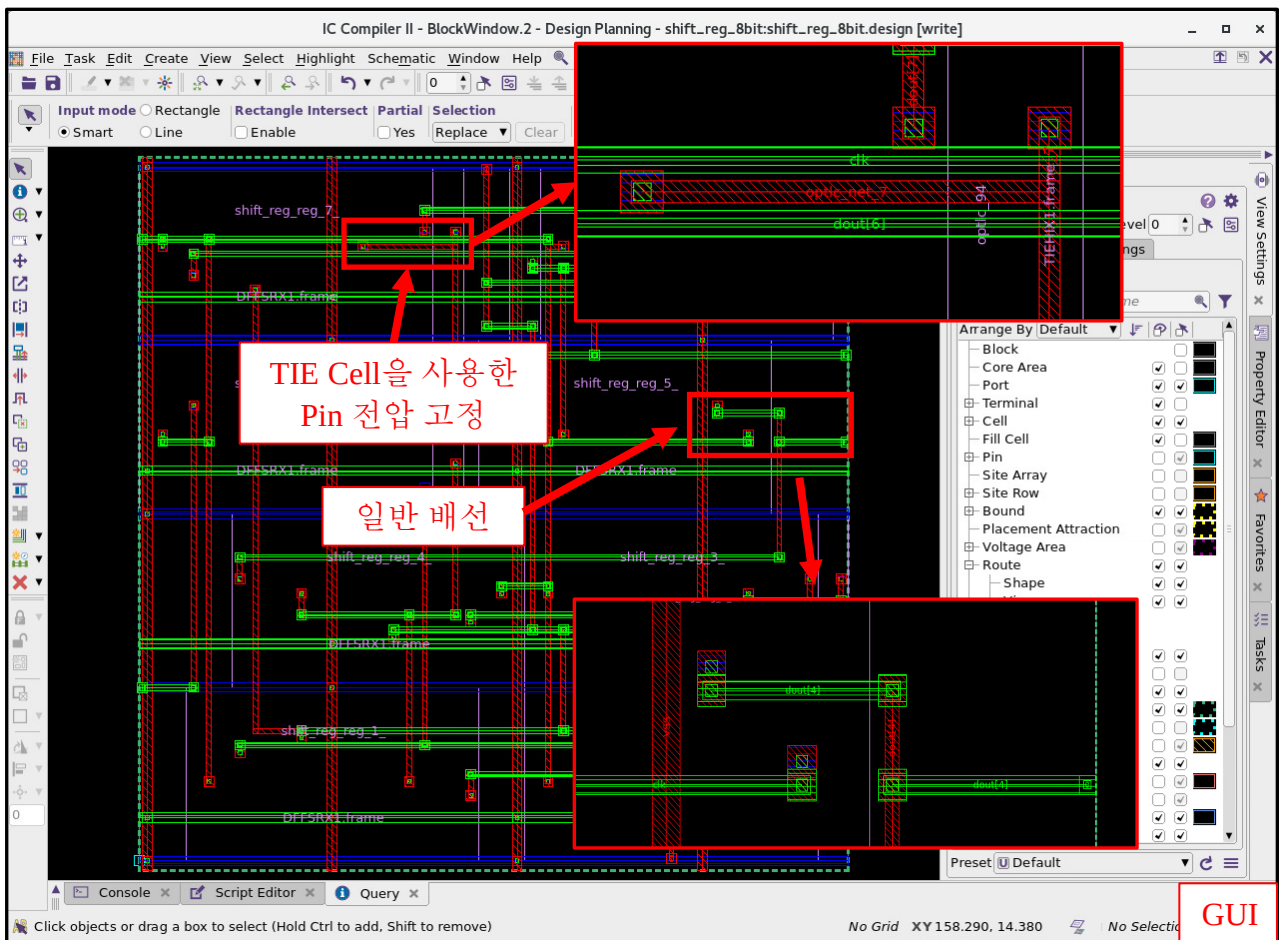
- Auto P&R 중 Auto Route 과정

① Auto Route

```
open_lib TOP_MODULE
copy_block -force -from_block TOP_MODULE_cts -to_block TOP_MODULE_route
open_block TOP_MODULE_route
route_global
route_track
route_detail -incremental true -initial_drc_from_input true
check_routes
optimize_routes
check_routes
route_opt
check_routes
check_lvs
report_cells [get_cells -hierarchical -filter "is_hierarchical == false"]
close_blocks -save
```

ICC2 Tool Command (icc2_cmd.tcl 중 일부)

- Clock 배선을 제외한 나머지 일반 배선 Auto Route 진행
- route_global command를 사용하여 전체적인 Net 연결 분석
- route_track command를 사용하여 Metal Track 맞게 Routing
- route_detail command를 사용하여 Desing Rule에 맞게 Routing
- check_routes command를 사용하여 지금까지 문제가 없었는지 확인
(문제가 있어도 이후 진행하는 route command를 통해서 대부분 해결됨)
- optimize_routes command를 사용하여 순수 물리적인 배선 개선
- route_opt command를 사용하여 Timing을 고려하여 배선 개선
- check_lvs command를 사용하여 배선 과정에서 문제가 없었는지 검증
- 마지막 check_routes와 check_lvs에서 문제가 있는 경우 route_detail 부터 다시 진행 (계속 문제가 발생시 update_timing 후 place_opt, legalize_placement 진행 후 route_global부터 진행)
- Timing 제약이 심하거나 Core Size가 작은 경우 문제가 계속 발생하기 때문에 해당 단계부터 다시 진행 (SDC File 수정하는 경우 Synthesis 부터 다시 진행하고 Core Size 수정은 Auto P&R File Load부터 다시 진행)



```

icc2_shell> check_lvs

=====
Maximum number of violations is set to 20
Abort checking when more than 20 violations are found
All violations might not be found.
=====

Total number of input nets is 17.
Total number of short violations is 0.
Total number of open nets is 0.
Total number of floating route violations is 0.
  
```

CLI Log

- GUI에서 Auto Route의 결과를 확인 가능
- GUI에서 TIE Cell을 사용하여 Floating Pin을 1또는 0으로 고정하여 Power에서 직접 연결하지 않는 것을 확인 가능
- CLI Log를 통해 Auto Route 결과 확인 가능
- Synthesis에서 확인한 report command 결과와 유사 (뒷장에서 계속)

DRC - SUMMARY:

@@@@@@@ TOTAL VIOLATIONS = 0

Violations = 0 인 것을 확인

```
Total Wire Length =                2083 micron
Total Number of Contacts =          99
Total Number of Wires =              83
Total Number of PtConns =            0
Total Number of Routed Wires =       83
Total Routed Wire Length =           2083 micron
Total Number of Routed Contacts =     99
  Layer    MET1 :                   1 micron
  Layer    MET2 :                   923 micron
  Layer    MET3 :                   1159 micron
  Via      V23 :                     55
  Via      V12 :                     44
```

check_routes 결과 중 일부

Start checking for open nets ...

Errors 0 인 것을 확인

```
Total number of nets = 17, of which 0 are not extracted
Total number of open nets = 0, of which 0 are frozen
```

Check 17 nets, 0 have Errors

check_routes 결과 중 일부

Verify Summary:

DRCs = 0 인 것을 확인

```
Total number of nets = 17, of which 0 are not extracted
Total number of open nets = 0, of which 0 are frozen
Total number of excluded ports = 0 ports of 0 unplaced cells connected to 0 nets
                                0 ports without pins of 0 cells connected to 0 nets
                                0 ports of 0 cover cells connected to 0 non-placed cells
Total number of DRCs = 0
Total number of antenna violations = no antenna rules defined
Total number of tie to rail violations = not checked
Total number of tie to rail directly violations =
```

check_routes 결과 중 일부

→ report command 결과를 통해서 Auto Route 결과 확인

5) Auto P&R 결과 저장

- Auto P&R 진행한 결과를 검증하기 위해 각 File 저장하는 과정

① Fill Cell 추가 및 Auto P&R 결과 저장

```
open_lib TOP_MODULE
copy_block -force -from block TOP_MODULE_route -to_block TOP_MODULE_fin
open_block TOP_MODULE_fin
create_stdcell_fillers -lib cells {{FILLX4}} {{FILLX3}} {{FILLX2}} {{FILLX1}}
create_terminal -port VDD -boundary {{0 8}} {{2 10}} -layer MET3
create_terminal -port VSS -boundary {{0 44}} {{2 46}} -layer MET3
write_gds -lib_cell_view frame -layer_map ../MAP/NSPL_0p5um_Digital_CMOS_2P3M_5V
write_verilog -exclude {physical_only_cells pg_netlist supply_statements} ../VER
write_parasitics -format spef -corner CORNER -output ../TLU_pnr/TOP_MODULE.spef
write_sdc -version 1.8 -corner CORNER -output ../SDC_pnr/TOP_MODULE.sdc
write_sdf ../SDF_pnr/TOP_MODULE.sdf
report_cells [get_cells -hierarchical -filter "is_hierarchical == false"]
report_qor
close_blocks -save
```

ICC2 Tool Command (icc2_cmd.tcl 중 일부)

```
shift_reg_8bit.gds
shift_reg_8bit.sdc
shift_reg_8bit.v
shift_reg_8bit.sdf
shift_reg_8bit.spef.NSPL_0p5um_Digital_CMOS_2P3M_5V_V1p1_12T std
```

저장한 File

- Cell과 Cell 사이 빈 공간을 채우기 위해 FILL Cell 삽입
- DRC 및 LVS 검증을 하기 위해 GDSII Format으로 GDS File 저장
- POST-STA를 진행하기 위해 Verilog (Gate Level Netlist) File 저장
- POST-STA를 진행하기 위해 SDC (Timing 제약) File 저장
- POST-STA를 진행하기 위해 배선 정보 (R/C/CC) 를 나타내는 SPEF (Standard Parasitic Exchange Format) File 저장
- POST-STA를 진행하기 위해 Place 된 Cell의 Timing, Pin Cap, Power 등 정보를 나타내는 SDF (Standard Delay Format) File 저장

6. POST-STA

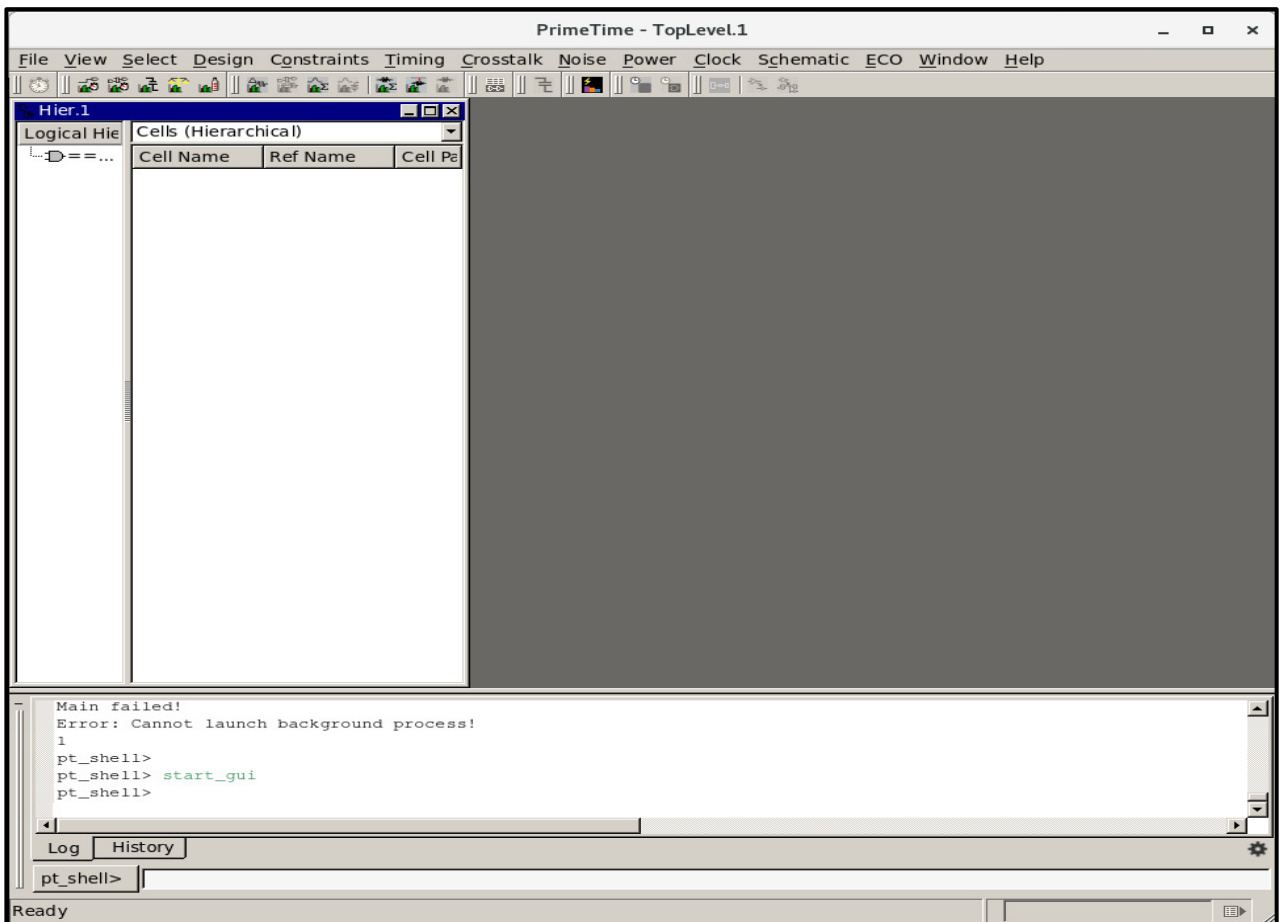
6. POST-STA

- 목적 : Auto P&R 검증

1) Auto P&R 후 POST-STA

- Auto P&R 결과 검증하는 과정

① PT(Prime Time) Tool 실행



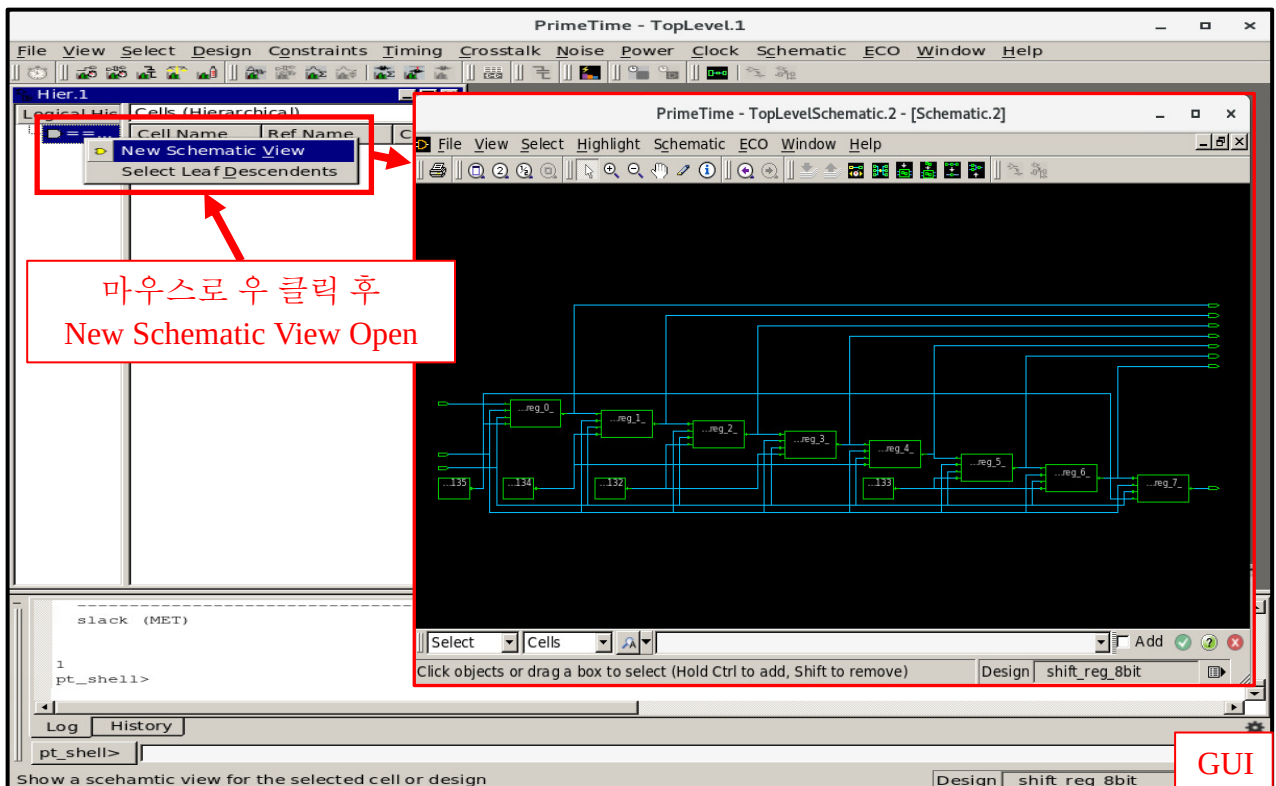
```
pt_shell> start_gui
```

- Tool : PT (Prime Time)
- 설명 : POST-STA 진행하는 Synopsys사의 Tool (/user/tool/path/Synopsys/PrimeTime/version/bin/pt_shell)
 - RUN Directory에서 /tool/path/pt_shell command로 실행
 - start_gui command를 통하여 GUI로 각 단계별 변화를 확인 가능
 - GUI로 진행하는 것이 아니라 CLI로 진행할 예정

② POST-STA를 위해 File Load 및 POST-STA 진행

```
read_db ../DB/DB_FILE.db
set_target_library [list ../DB/DB_FILE.db]
set_link_library [list * ../DB/DB_FILE.db]
read_verilog ../VERILOG_pnr/TOP_MODULE.v
current_design TOP_MODULE
read_parasitics ../TLU_pnr/TOP_MODULE.spf.NSPL_0p5um_Digital_CMOS_2P3M_5V_V1p
read_sdc -version 1.8 ../SDC_pnr/TOP_MODULE.sdc
read_sdf ../SDF_pnr/TOP_MODULE.sdf
report_analysis_coverage
report_constraint -all_violators
report_timing -delay_type max
report_timing -delay_type min
report_global_timing
report_qor
```

PT Tool Command 중 일부



GUI

- Auto P&R에서 저장한 File Load
- GUI에서 Synthesis를 통해 얻은 Gate Level Netlist와 달라진 점을 Schematic으로 확인 가능
- report command를 사용하여 POST-STA 진행 (뒷장 참고)

```

Startpoint: shift_reg_reg_0_
(rising edge-triggered flip-flop clocked by clk)
Endpoint: dout[0] (output port clocked by clk)
Path Group: clk
Path Type: max

```

Point	Incr	Path
clock clk (rise edge)	0.00	0.00
clock network delay (propagated)	0.00	0.00
shift_reg_reg_0_/CLK (DFFSRX1)	0.00	0.00 r
shift_reg_reg_0_/Q (DFFSRX1)	2.45 *	2.45 f
dout[0] (out)	0.00 *	2.45 f
data arrival time		
		2.45
clock clk (rise edge)	20.00	20.00
clock network delay (ideal)	0.00	20.00
clock reconvergence pessimism	0.00	20.00
clock uncertainty	-0.50	19.50
output external delay	-6.00	13.50
data required time		
		13.50
data required time		13.50
data arrival time		-2.45
slack (MET)		11.05

Slack 값이 양수가 나오면 Pass
음수가 나오면 Fail

report_timing 결과 중 일부

```

*****
Report : constraint
       -all_violators
       -verbose
Design : shift_reg_8bit
Version: K-2015.06-SP3-1
Date   : Thu Aug  6 15:49:07 2026
*****

```

내용이 없으면 Pass

report_timing 결과 중 일부

Type of Check	Total	Met	Violated	Not Met
setup	24	8 (33%)	0 (0%)	16 (67%)
hold	24	8 (33%)	0 (0%)	16 (67%)
recovery	16	0 (0%)	0 (0%)	16 (100%)
removal	16	0 (0%)	0 (0%)	16 (100%)
min_pulse_width	32	16 (50%)	0 (0%)	16 (50%)
out_setup	8	8 (100%)	0 (0%)	0 (0%)
out_hold	8	8 (100%)	0 (0%)	0 (0%)
All Checks	128	48 (38%)	0 (0%)	80 (63%)

총 합 100%면 Pass
(반올림으로 인해 오차 1%)

report_analysis_coverage
결과 중 일부

```

Report : qor
Design : shift_reg_8bit
Version: K-2015.06-SP3-1
Date   : Thu Aug  6 15:50:08 2026
*****

```

report_qor
결과 중 일부

```

Timing Path Group 'clk' (max_delay/setup)
-----
Levels of Logic:                1
Critical Path Length:           2.45
Critical Path Slack:             11.05
Total Negative Slack:            0.00
No. of Violating Paths:          0
-----

```

Timing Path Group 'clk' (min_delay/

```

-----
Levels of Logic:
Critical Path Length:
Critical Path Slack:            0.38
Total Negative Slack:           0.00
No. of Violating Paths:         0
-----

```

Cell Count

```

-----
Hierarchical Cell Count:        0
Hierarchical Port Count:        0
Leaf Cell Count:                 12
-----

```

report_qor
결과 중 일부

2) Auto P&R 후 Netlist Function 검증

- Auto P&R 후 Netlist 변경(Cell 추가 및 제거)으로 Function 문제 없는지 확인하는 과정

① Test Bench 수정

```
`timescale 1ns/1ps

module tb_shift_reg_8bit;

    reg        clk;
    reg        rst_n;
    reg        din;
    wire [7:0] dout;

    shift_reg_8bit uut (
        .clk      (clk),
        .rst_n    (rst_n),
        .din      (din),
        .dout     (dout)
    );

    always #10 clk = ~clk;

initial begin
    clk      = 1'b0;
    rst_n    = 1'b0;
    din      = 1'b0;

    $display("=====");
    $display("  RTL-Only 8-bit Shift Register Sim  ");
    $display("=====");

    #25;
    rst_n = 1'b1;
    @(posedge clk);

    @(negedge clk);
    din = 1'b1;
    @(posedge clk); #1;
    $display("Time=%0t ps | din=%b -> dout=%0d (Binary: %b)", $time,
```

제공 받은 Test Bench File
(shift_reg_8bit_tb.v 중 일부)

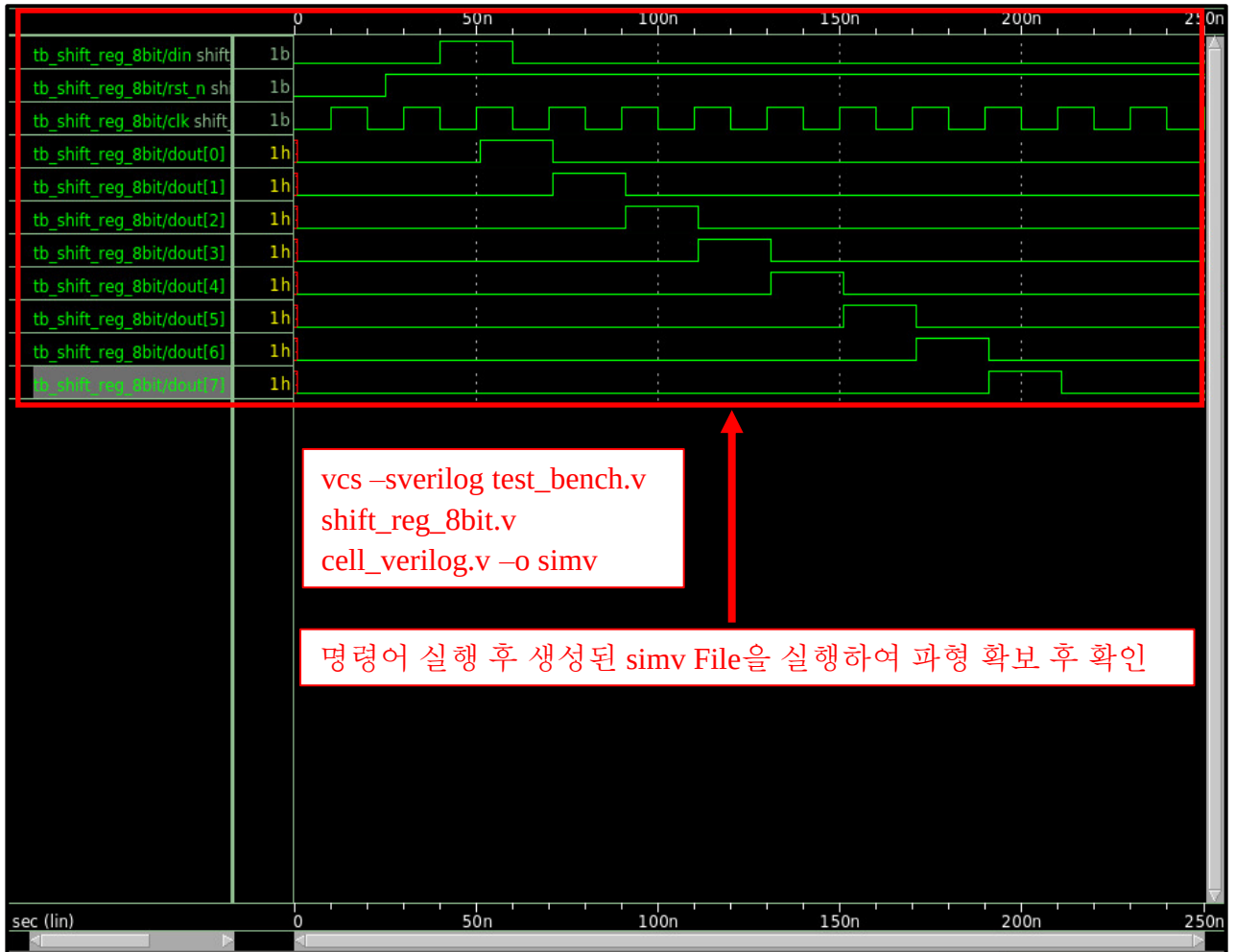
Input, Output Pin 설정

Clock 주기 설정

Clock에 따른 Signal 설정

→ Gate Level Netlist 검증에서 사용한 Test Bench File 사용

② VCS Tool을 사용한 Auto P&R 후 변경된 Netlist Function Check

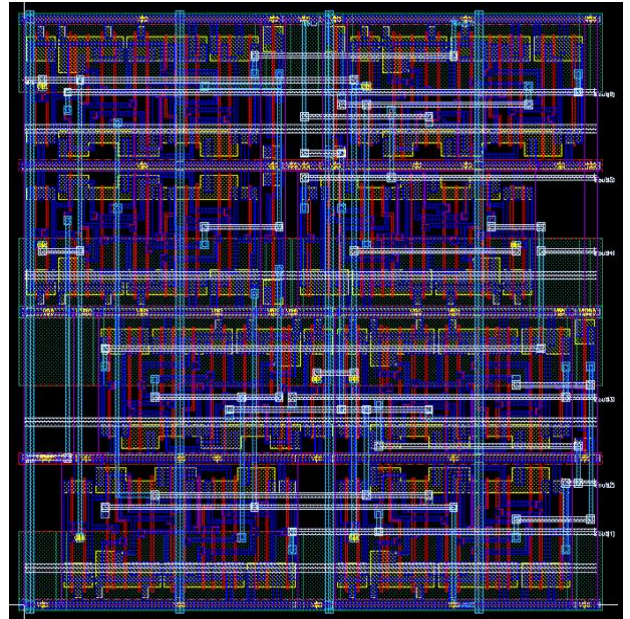
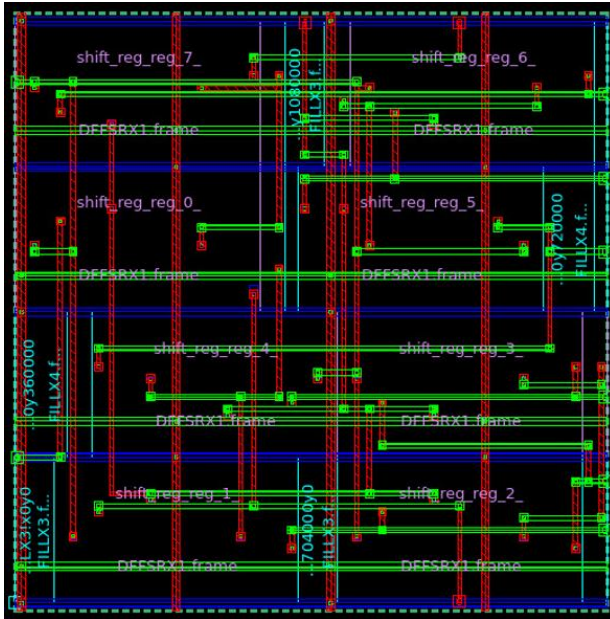


- 제공된 Standard Cell Function이 기술된 Verilog도 포함되어 진행
- 원하는 파형이 아닌 경우 Test Bench File 수정하여 다시 진행
- Test Bench 설정이 문제 없으면 POST-STA부터 Synthesis까지 역방향으로 하나씩 확인하며 문제가 있는 곳부터 다시 진행 (Core Size, Timing 제약 등 수정이 필요한 수정 후 해당 단계부터 다시 진행)

2) Auto P&R 후 DRC, LVS 검증

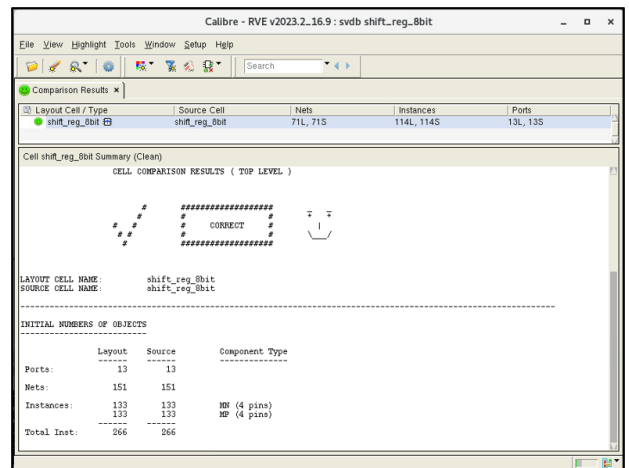
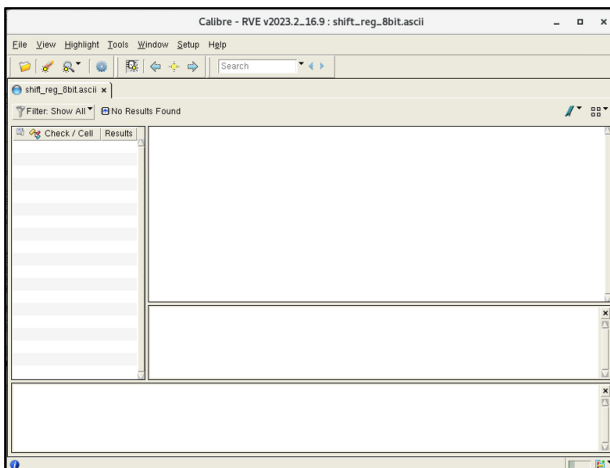
- Auto P&R에서 검증하지 못하는 DRC, LVS 항목을 검증하는 과정

① GDS 확인



→ ICC2 Tool에서 확인한 Core의 모습과 Virtuoso Tool에서 확인한 Core 모습

② DRC 및 LVS



→ DRC 및 LVS 결과 확인 (LVS에서 사용하는 Source File 확보는 뒷장 설명 참고)

```

$ Spice netlist generated by v2lvs
$ v2023.2_16.9   Tue Apr 4 17:40:20 PDT 2023
*.BUSDELIMITER [
.include ./NSPL_0p5um_Digital_CMOS_2P3M_5V_V1p1_12T_std.cdl
.SUBCKT shift_reg_8bit clk rst_n din dout[7] dout[6] dout[5] dout[4] dout[3]
+ dout[2] dout[1] dout[0]
Xshift_reg_reg_7_ DFFSRX1 $PINS D=dout[6] CLK=clk R=rst_n S=optlc_net_6
+ Q=dout[7]
Xshift_reg_reg_6_ DFFSRX1 $PINS D=dout[5] CLK=clk R=rst_n S=optlc_net_6
+ Q=dout[6]
Xshift_reg_reg_5_ DFFSRX1 $PINS D=dout[4] CLK=clk R=rst_n S=optlc_net_9
+ Q=dout[5]
Xshift_reg_reg_4_ DFFSRX1 $PINS D=dout[3] CLK=clk R=rst_n S=optlc_net_9
+ Q=dout[4]
Xshift_reg_reg_3_ DFFSRX1 $PINS D=dout[2] CLK=clk R=rst_n S=optlc_net_8
+ Q=dout[3]
Xshift_reg_reg_2_ DFFSRX1 $PINS D=dout[1] CLK=clk R=rst_n S=optlc_net_5
+ Q=dout[2]
Xshift_reg_reg_1_ DFFSRX1 $PINS D=dout[0] CLK=clk R=rst_n S=optlc_net_7
+ Q=dout[1]
Xshift_reg_reg_0_ DFFSRX1 $PINS D=din CLK=clk R=rst_n S=optlc_net_7 Q=dout[0]
Xoptlc_138 TIEHIX1 $PINS Y=optlc_net_5
Xoptlc_139 TIEHIX1 $PINS Y=optlc_net_6
Xoptlc_140 TIEHIX1 $PINS Y=optlc_net_7
Xoptlc_141 TIEHIX1 $PINS Y=optlc_net_8
Xoptlc_142 TIEHIX1 $PINS Y=optlc_net_9
.ENDS
.GLOBAL VDD
.GLOBAL VSS

```

전달 받은 TOP_CDL File을
Include하는 구문 추가

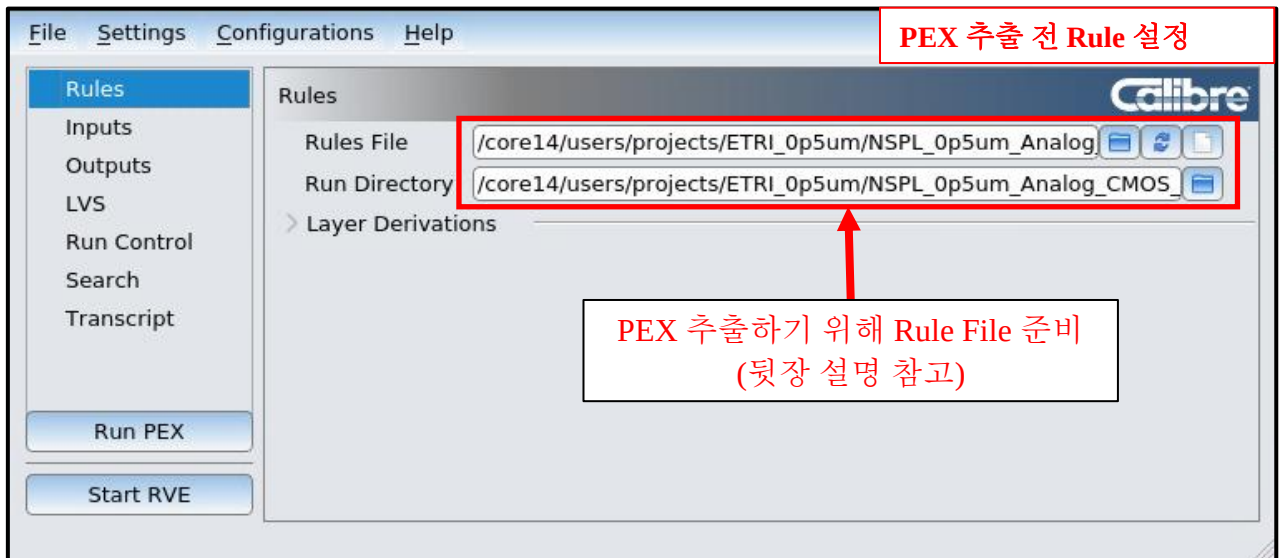
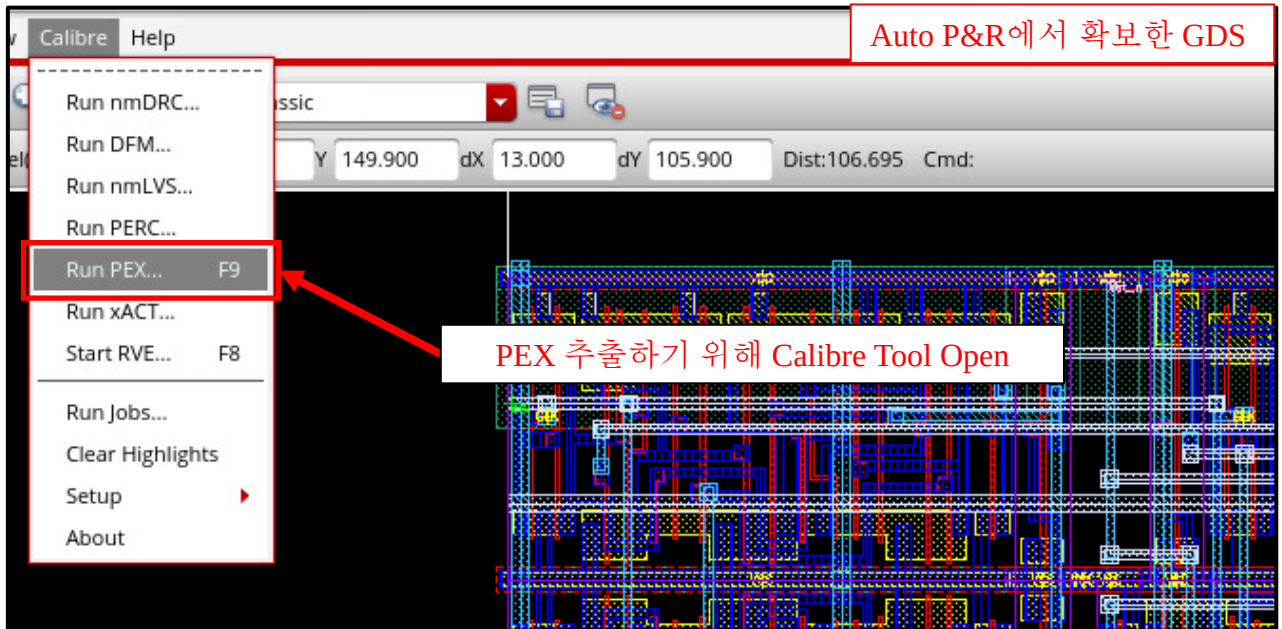
```
cmd : v2lvs -v netlist.v -s1 VDD -s0 VSS -lsp ../CDL/TOP_CDL.cdl -o TOP_MODULE.cdl
```

- Tool : v2lvs (Verilog to LVS Source File)
- 설명 : Verilog File을 LVS 용 Source File로 변환해주는 Siemens사 Tool (calibre와 공유 되어 v2lvs 바로 사용 가능)
- Auto P&R에서 저장한 Netlist.v가 있는 Directory에서 실행
- LVS및 PEX 추출 과정에서 사용할 Source(CDL) File을 확보하기 위해서 v2lvs 진행
- v2lvs -v netlist.v -s1 VDD -s0 VSS -lsp ../CDL/TOP_CDL.cdl -o TOP_MODULE.cdl command 사용하여 Source File 확보
- 확보한 CDL File에서 TOP_CDL Include 구문 추가

4) PEX 추출 및 Simulation 진행

- Auto P&R 결과물을 기반으로 Simulation 진행하여 성능 검토하는 과정

① PEX 추출 및 Simulation 진행



- Auto P&R 후 저장한 GDS File을 Cadence 사의 Virtuoso Layout Editor Tool에 Import 하여 해당 GDS의 Layout View Open
- PEX 추출하기 위해서 Calibre Tool Open 및 전달 받은 PEX 추출 용 Rule File 기입 할 예정 (뒷장 계속)

```
#DEFINE calibreXRC
```

pex_header.cal File

```
#DEFINE TYPICAL
```

```
// #DEFINE WROST
```

```
// #DEFINE BEST
```

```
LAYOUT PRIMARY "TOPCELL"
```

```
LAYOUT PATH "TOPCELL.gds"
```

```
LAYOUT SYSTEM GDSII
```

```
SOURCE PRIMARY "TOPCELL"
```

```
SOURCE PATH "TOPCELL.cdl"
```

```
SOURCE SYSTEM SPICE
```

```
LVS REPORT "lvs.rep"
```

```
LVS REPORT MAXIMUM 1000 // ALL
```

```
LVS REPORT OPTION A B C D S V
```

```
ERC RESULTS DATABASE "erc.db" ASCII
```

```
ERC SUMMARY REPORT "erc.sum"
```

```
ERC MAXIMUM RESULTS 100
```

ETRI_05um_CMOS_PDK에서 PEX 항목 중
다음 3개의 File 및 Directory를 RUN Directory로 Copy

LPE_LIB

NSPL_0p5um_Analog_CMOS_2P3M_5V_V1.1_PEX.cal

pex_header.cal

Copy한 PEX Rule File이 있는
Directory Path로 변경

```
INCLUDE "/your/path/NSPL_0p5um_Analog_CMOS_2P3M_5V_V1.1_PEX.cal"
```

```
#IFDEF TYPICAL
```

```
INCLUDE "/your/path/LPE_LIB/typical_rules.C"
```

```
INCLUDE "/your/path/LPE_LIB/typical_rules.R"
```

```
#ENDIF
```

```
#IFDEF BEST
```

```
INCLUDE "/your/path/LPE_LIB/bestRC_rules.C"
```

```
INCLUDE "/your/path/LPE_LIB/bestRC_rules.R"
```

```
#ENDIF
```

```
#IFDEF WORST
```

```
INCLUDE "/your/path/LPE_LIB/worstRC_rules.C"
```

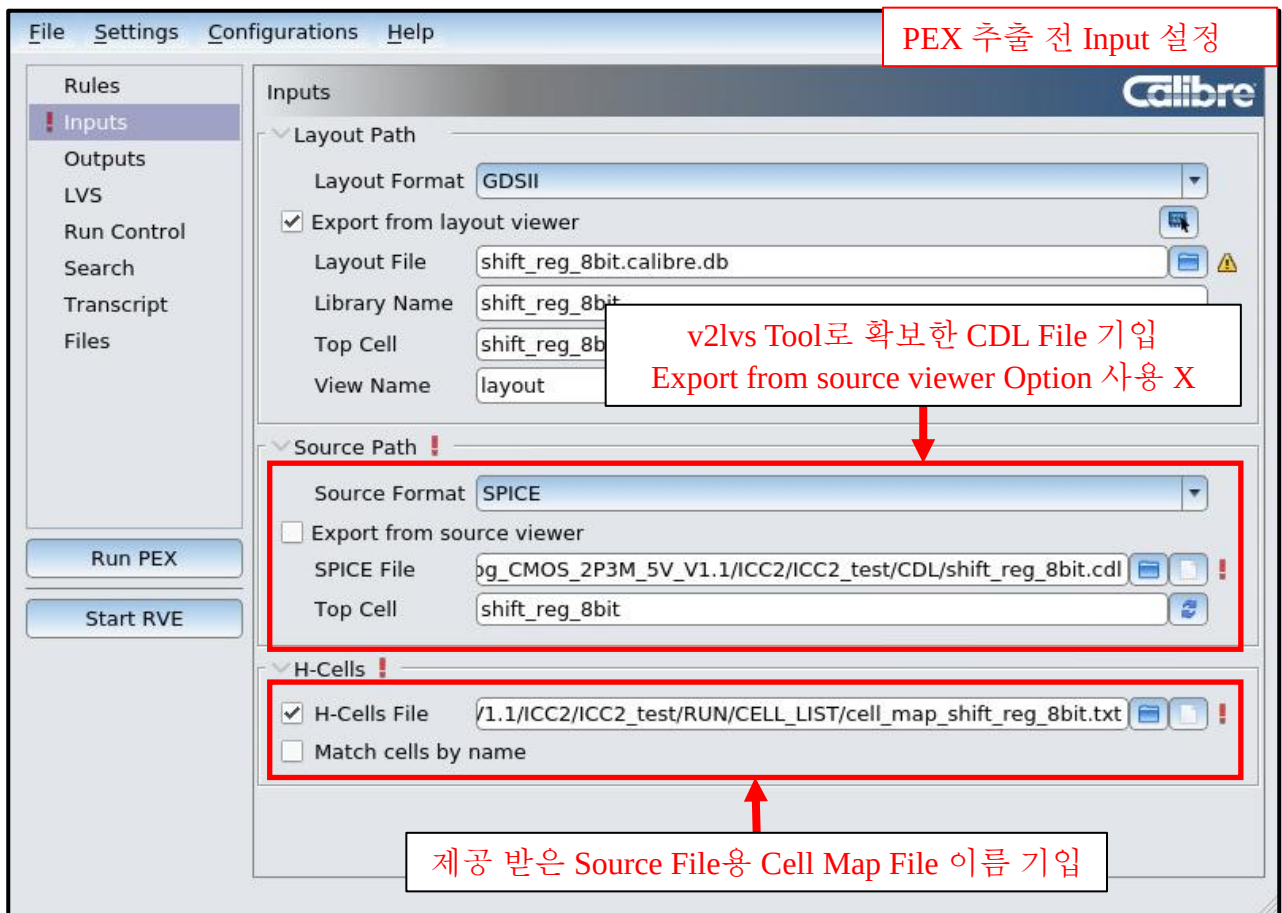
```
INCLUDE "/your/path/LPE_LIB/worstRC_rules.R"
```

```
#ENDIF
```

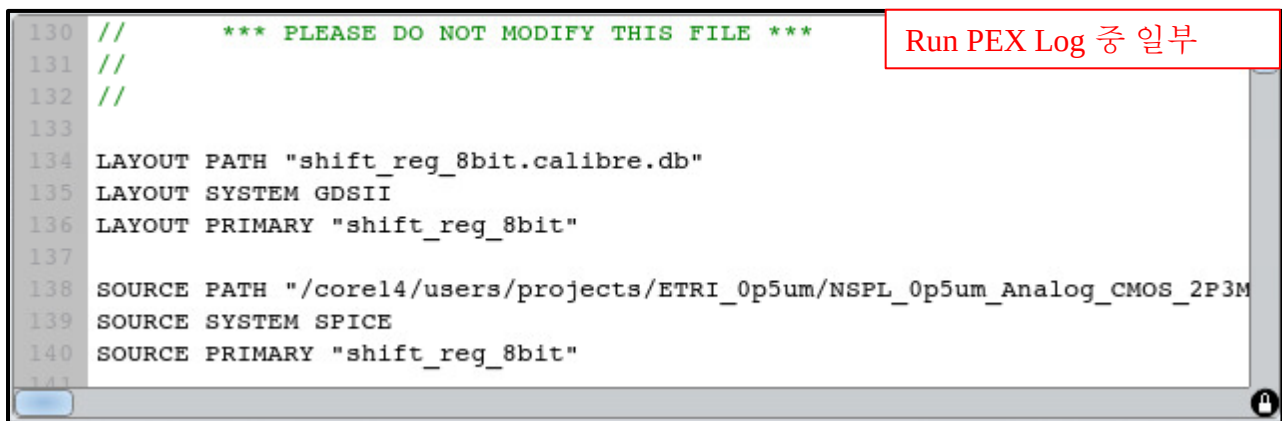
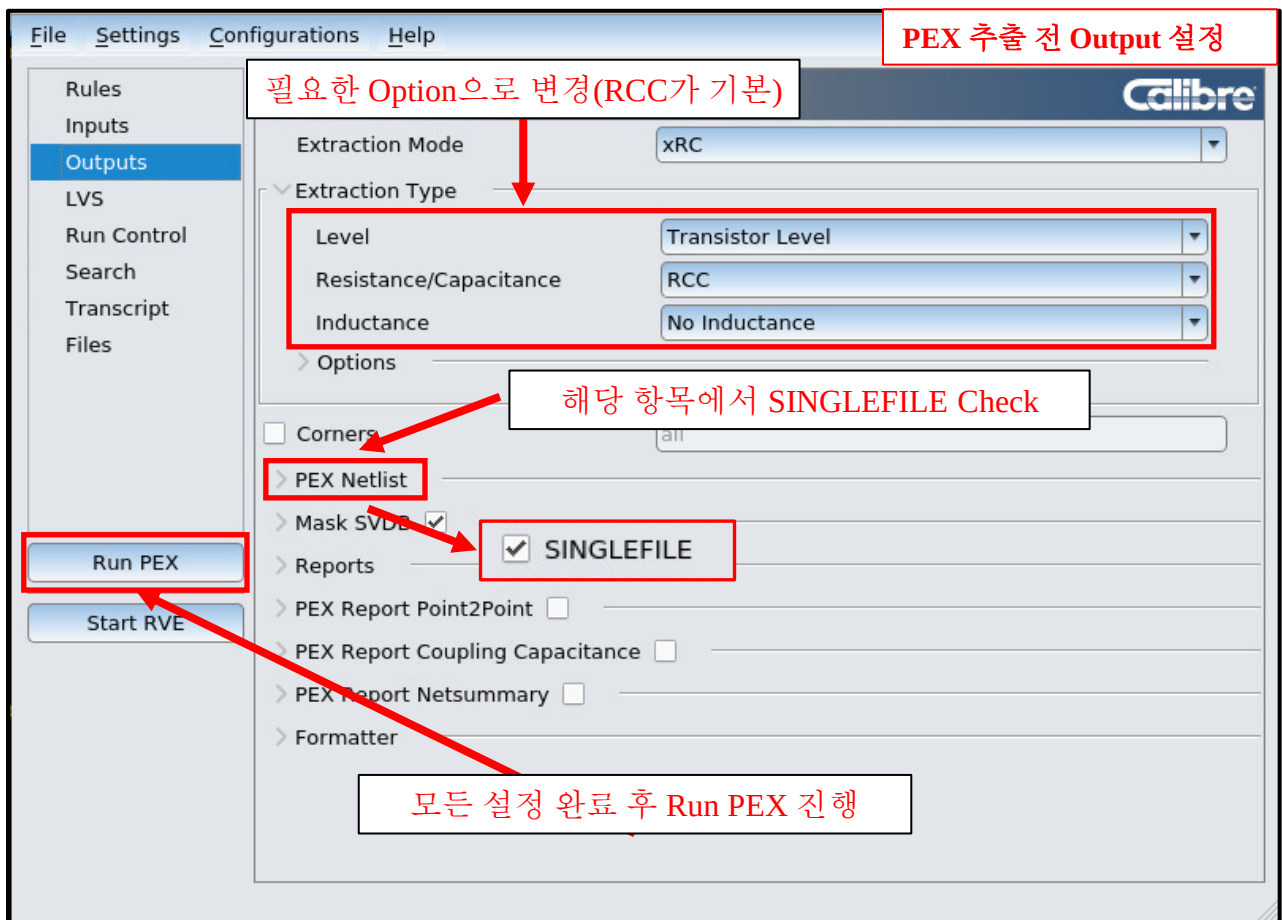
PEX.cal File 중 일부

Copy한 LPE_LIB Directory
Path로 변경

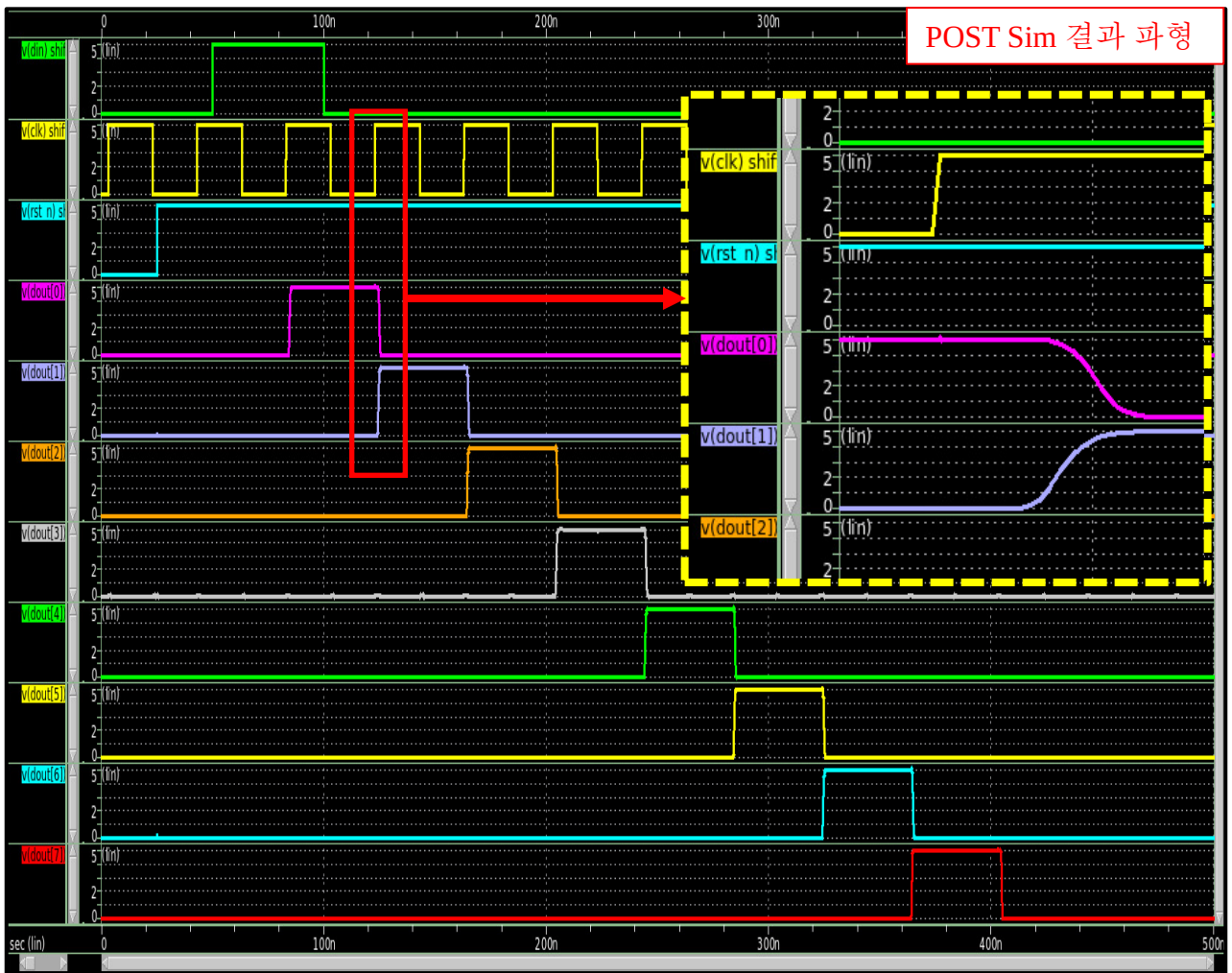
- ETRI_05_CMOS_PDK에서 PEX 항목 중 일부를 RUN Directory로 Copy
- Copy한 File 중 header.cal과 PEX.cal File 내부 INCLUDE Path 변경
- Copy 및 변경한 File 중 header.cal File을 /your/path/header.cal 형식으로 Rules File 항목에 기입
- header.cal File이 있는 경로를 Run Directory 항목에 기입 (/your/path 형식으로 마지막 "/" 가 없어야 함)



- Layout Path는 Layout View에서 Tool Open 시 자동으로 해당 Cell로 설정이 됨
- Source Path는 v2lvs Tool로 확보한 CDL File 사용
- H-Cells는 제공 받은 NSPL_0p5um_Digital_CMOS_2P3M_5V_V1p1_12T_std.map File에서 TOP_MODULE을 수정하고 해당 File 기입 (필요시 File을 Copy하여 사용)



- PEX 추출 전 Rule, Input, Output Page Option 수정
- Auto P&R에서 Place 된 Cell이 많은 경우 Extraction Type에서 Resistance/Capacitance 항목을 RCC가 아닌 RC 또는 R 항목으로 변경 (RC, RCC 항목으로 진행하면 이후 Simulation 진행 시간이 증가함)
- 모든 항목 검토 후 Run PEX 진행



- 추출한 PEX File(TOP_MODULE.pex.netlist File)을 기반으로 Simulation 진행 (Simulation은 익숙한 Tool을 사용하여 VCS Tool을 사용한 Function Check Test Bench 참고하여 진행)
- 각 Input과 Clock, Output 파형을 확인하여 문제가 없는지 검증
- 사용하는 Cell의 양, Simulation 조건, Simulation Tool마다 속도가 다르기 때문에 주의해야 함
- PEX를 기반으로 진행한 Simulation에서 Function 문제가 없고 원하는 성능을 만족하는 검토
- 모든 문제가 없고 성능에 만족하면 RTL을 기반으로 Auto P&R까지 진행하는 Basic Tutorial은 완료한 것으로 진행하면서 제작한 File을 정리하여 다른 과정에서 사용하면 됨